Astra Control Service를 사용합니다

Astra Control Service

NetApp
January 31, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/astra-control-service/use/log-in.html on January 31, 2022. Always check docs.netapp.com for the latest.

목차

Astra Control Service를 사용합니다 1
Astra Control Service에 로그인합니다 1
애플리케이션 관리 및 보호 1
앱 및 컴퓨팅 상태 보기 33
버킷을 관리합니다 36
계정을 관리합니다 39
앱 및 클러스터 관리를 취소합니다 48

Astra Control Service를 사용합니다

Astra Control Service에 로그인합니다

Astra Control Service는 로 이동하여 SaaS 기반 사용자 인터페이스를 통해 액세스할 수 있습니다 https://astra.netapp.io.

> Single Sign-On을 사용하여 회사 디렉터리(통합 ID)의 자격 증명을 사용하여 로그인할 수 있습니다. 자세한 내용은 로 이동하십시오 "Cloud Central 도움말 센터" 그런 다음 * Cloud Central 로그인 옵션 * 을 선택합니다.

무엇을 '필요로 할거야

- "Cloud Central 사용자 ID입니다".
- "새로운 Astra Control 계정" 또는 .
- 지원되는 웹 브라우저입니다.

  Astra Control Service는 최신 버전의 Firefox, Safari 및 Chrome을 최소 1280 x 720의 해상도로 지원합니다.

단계

1. 웹 브라우저를 열고 로 이동합니다 https://astra.netapp.io.
2. NetApp Cloud Central 자격 증명을 사용하여 로그인합니다.

애플리케이션 관리 및 보호

앱 관리를 시작합니다

먼저 해 "Astra Control에 Kubernetes 클러스터를 추가합니다", 클러스터(Astra Control 외부)에 앱을 설치한 다음 Astra Control의 앱 페이지로 이동하여 앱 관리를 시작할 수 있습니다.

설명합니다

다음과 같은 Astra Control 앱 관리 요구 사항을 고려하십시오.

- * 라이선스 *: Astra Control Center를 사용하여 앱을 관리하려면 Astra Control Center 라이센스가 필요합니다.
- * Namespaces *: Astra Control은 앱이 단일 네임스페이스 이상의 범위를 포괄하지 않도록 하지만 네임스페이스에는 여러 개의 앱이 포함될 수 있습니다.
- * StorageClass *: StorageClass가 명시적으로 설정된 앱을 설치하고 앱을 복제해야 하는 경우 클론 작업의 타겟 클러스터에 원래 지정된 StorageClass가 있어야 합니다. 명시적으로 StorageClass를 동일한 StorageClass가 없는 클러스터로 설정한 애플리케이션을 클론 복제하면 실패합니다.
- * Kubernetes 리소스 *: Astra Control에서 수집하지 않은 Kubernetes 리소스를 사용하는 앱에는 전체 앱 데이터 관리 기능이 없을 수 있습니다. Astra Control은 다음과 같은 Kubernetes 리소스를 수집합니다.
  - 클러스터 역할
  - ClusterRoleBinding 을 참조하십시오

- ConfigMap을 클릭합니다
- 사용자 지정 리소스 정의
- CustomResource 를 선택합니다
- DemonSet
- 구축
- DeploymentConfig(배포 구성
- 침투
- 뮤atingWebhook
- PersistentVolumeClaim
- 포드
- ReplicaSet입니다
- RoleBinding 을 클릭합니다
- 역할
- 루트
- 비밀
- 서비스
- 서비스 계정
- StatefulSet 을 선택합니다
- Webhook을 확인합니다

지원되는 앱 설치 방법

다음 방법을 사용하여 Astra Control로 관리하려는 앱을 설치할 수 있습니다.

- * 매니페스트 파일 *: Astra Control은 kubctl을 사용하여 매니페스트 파일에서 설치된 앱을 지원합니다. 예를 들면 다음과 같습니다.

  ```
  kubectl apply -f myapp.yaml
  ```

- * Helm 3 *: Helm을 사용하여 앱을 설치하는 경우 Astra Control에 Helm 버전 3이 필요합니다. Helm 3(또는 Helm 2에서 Helm 3으로 업그레이드)과 함께 설치된 앱의 관리 및 클론 생성이 완벽하게 지원됩니다. Helm 2가 설치된 앱 관리는 지원되지 않습니다.
- * 운용자 구축 앱 *: Astra Control은 네임스페이스 범위 연산자와 함께 설치된 앱을 지원합니다. 이러한 연산자는 일반적으로 "pass-by-reference" 아키텍처가 아니라 "pass-by-value"로 설계되었습니다. 다음은 이러한 패턴을 따르는 일부 운영자 앱에 대한 설명입니다.
  - "아파치 K8ssandra"
  - "젠킨스 CI"
  - "Percona XtraDB 클러스터"

Astra Control은 "pass-by-reference" 아키텍처(예: CockroachDB 운영자)로 설계된 운영자를 복제하지 못할 수 있습니다. 이러한 유형의 클론 복제 작업 중에 클론 복제 운영자는 클론 복제 프로세스의 일부로 고유한 새로운 암호가 있음에도 불구하고 소스 운영자의 Kubernetes 암호를 참조하려고 합니다. Astra Control이 소스 운영자의 Kubernetes 암호를 모르기 때문에 클론 작업이 실패할 수 있습니다.

> 운영자와 설치하는 앱은 동일한 네임스페이스를 사용해야 합니다. 운영자가 배포 .YAML 파일을 수정해야 할 수도 있습니다.

클러스터에 앱을 설치합니다

클러스터를 Astra Control에 추가했으므로 클러스터에 앱을 설치할 수 있습니다. 영구 볼륨은 기본적으로 새 스토리지 클래스에 프로비저닝됩니다. Pod가 온라인 상태가 되면 Astra Control을 사용하여 앱을 관리할 수 있습니다.

Astra Control은 스토리지가 Astra Control에서 설치한 스토리지 클래스에 있는 경우에만 상태 저장 앱을 관리합니다.

- "GKE 클러스터용 저장소 클래스에 대해 알아봅니다"
- "AKS 클러스터용 스토리지 클래스에 대해 알아보십시오"

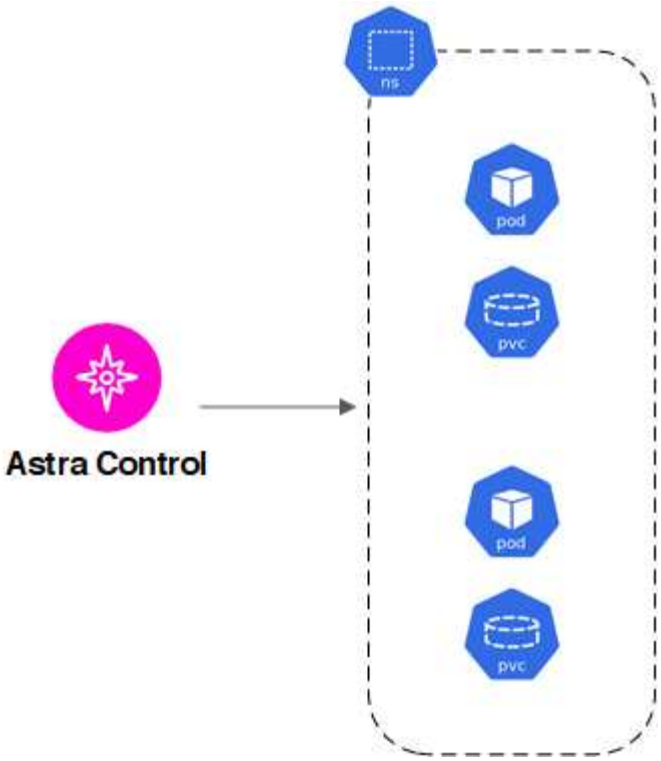
제어 차트에서 일반적인 응용 프로그램을 배포하는 방법에 대한 도움말은 다음을 참조하십시오.

- "제어 차트에서 MariaDB를 배포합니다"
- "제어 차트에서 MySQL을 배포합니다"
- "제어 차트에서 Postgres를 배포합니다"
- "제어 차트에서 Jenkins를 배포합니다"

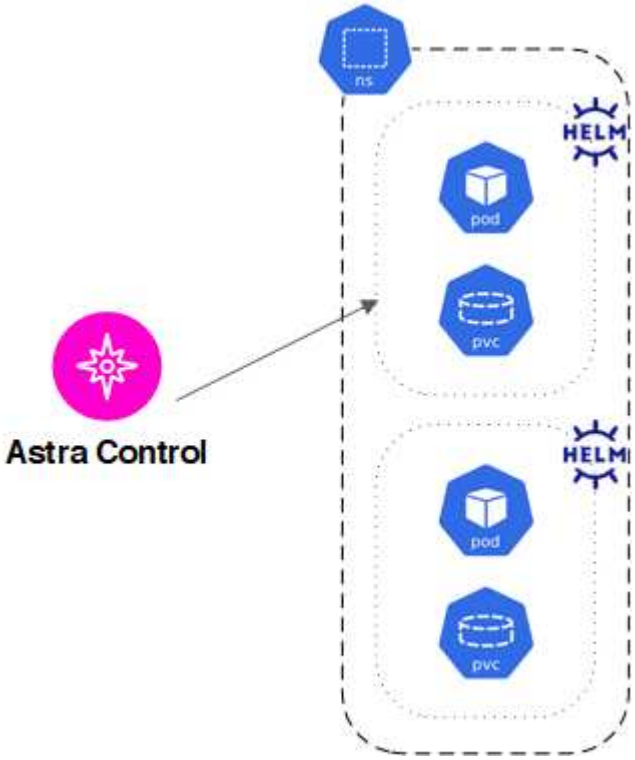
앱 관리

Astra Control은 클러스터에서 실행 중인 앱을 검색할 때 관리 방법을 선택할 때까지 관리되지 않습니다. Astra Control에서 관리되는 응용 프로그램은 다음 중 하나일 수 있습니다.

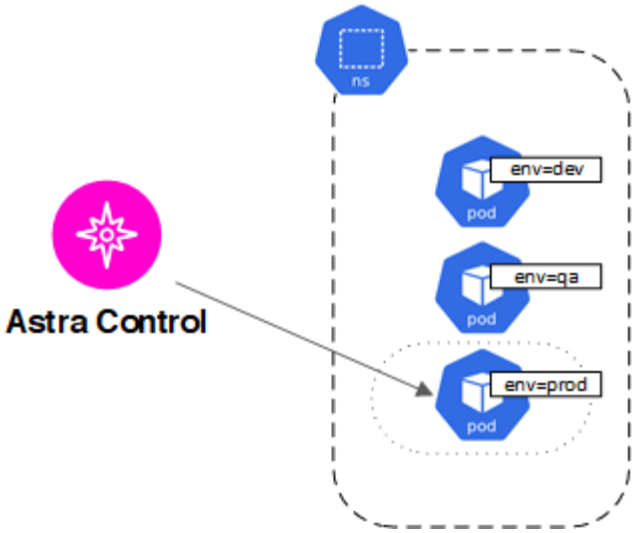
- 네임스페이스에서 모든 리소스를 포함하는 네임스페이스입니다



- 네임스페이스 내에서 helm3를 사용하여 배포된 개별 응용 프로그램입니다



- Kubernetes 레이블로 식별되는 리소스 그룹(Astra Control에서 _custom app_이라고 함)



아래 섹션에서는 이러한 옵션을 사용하여 앱을 관리하는 방법에 대해 설명합니다.

네임스페이스로 앱 관리

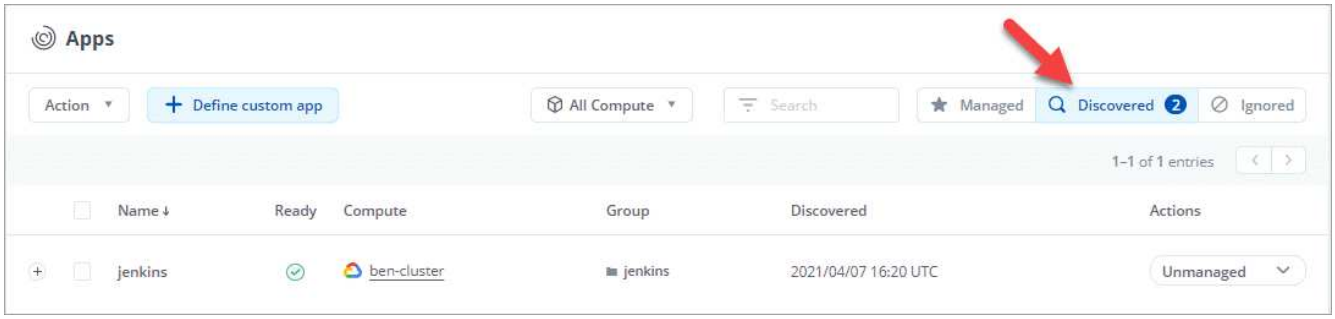
앱 페이지의 * 검색됨 * 섹션에는 네임스페이스와 Helm이 설치한 앱 또는 해당 네임스페이스의 사용자 지정 레이블 앱이 표시됩니다. 각 앱을 개별적으로 또는 네임스페이스 수준에서 관리하도록 선택할 수 있습니다. 데이터 보호 작업에 필요한 세분화 수준으로 세분화됩니다.

예를 들어 주 단위 주기를 가진 "Maria"에 대한 백업 정책을 설정할 수 있지만, "MariaDB"(동일한 이름 공간에 있음)를 더 자주 백업해야 할 수 있습니다. 이러한 요구사항에 따라 단일 네임스페이스가 아닌 앱을 별도로 관리해야 합니다.

Astra Control을 사용하면 계층 구조의 수준(네임스페이스 및 해당 네임스페이스의 앱)을 모두 별도로 관리할 수 있지만, 가장 좋은 방법은 하나 또는 다른 수준을 선택하는 것입니다. 작업이 네임스페이스 및 앱 수준에서 동시에 발생하면 Astra Control에서 수행하는 작업이 실패할 수 있습니다.

단계

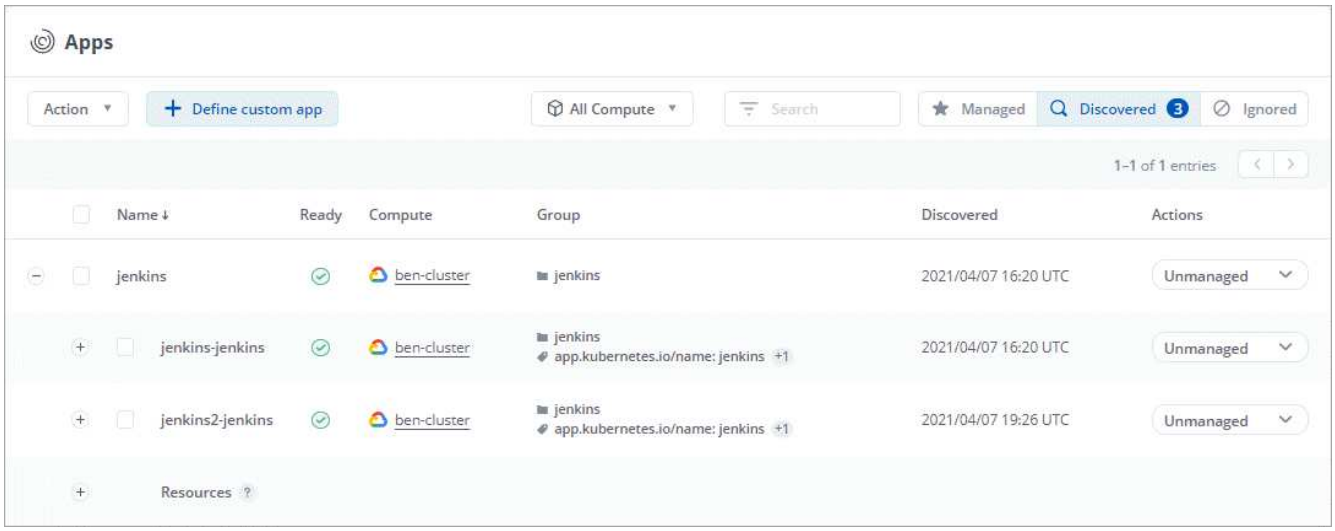
1. 응용 프로그램 * 을 선택한 다음 * 검색됨 * 을 선택합니다.



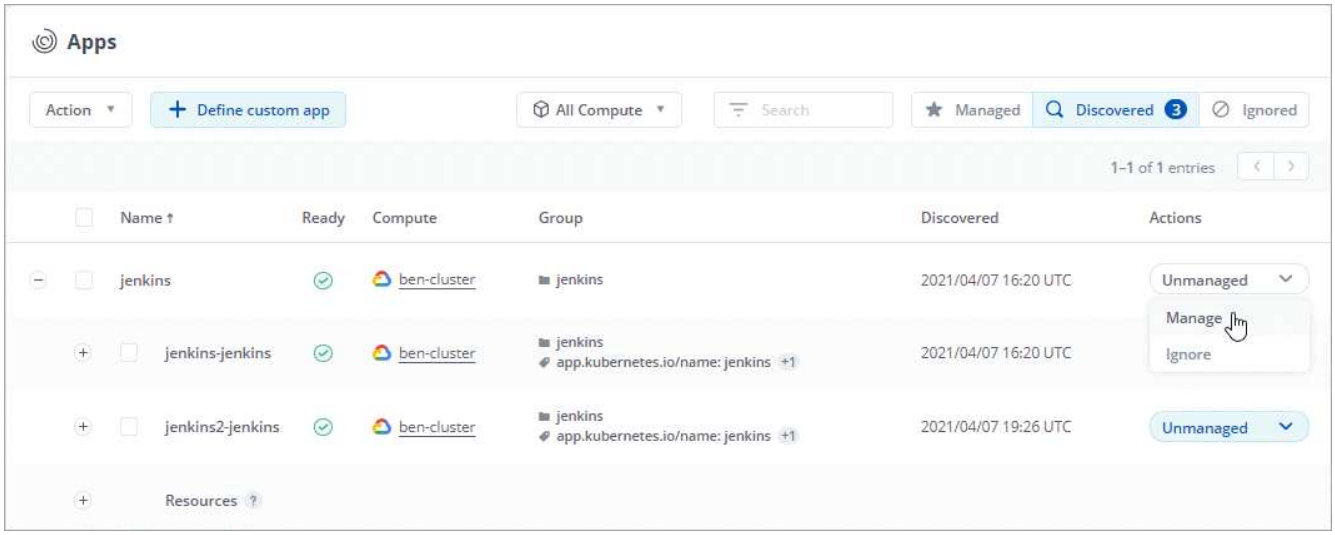
2. 검색된 네임스페이스 목록을 보고 네임스페이스를 확장하여 앱 및 관련 리소스를 봅니다.

   Astra Control은 Helm 앱 및 사용자 지정 레이블 앱을 네임스페이스에서 보여 줍니다. H제어 레이블을 사용할 수 있는 경우 태그 아이콘으로 지정됩니다.

   다음은 네임스페이스에서 두 개의 앱을 사용하는 예입니다.

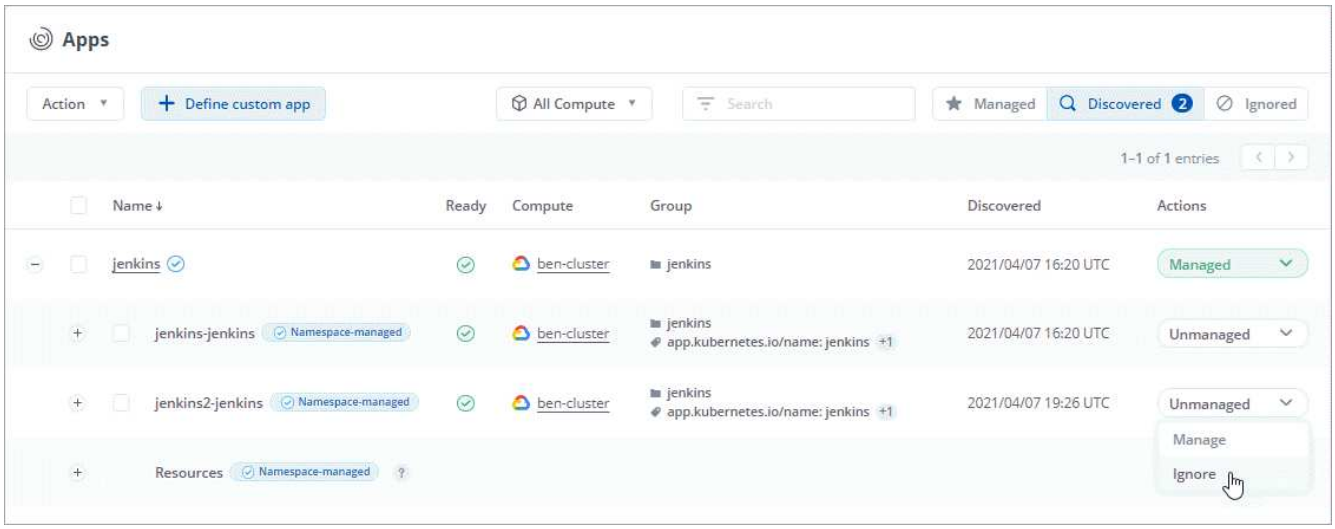


3. 각 앱을 개별적으로 관리할지 아니면 네임스페이스 수준에서 관리할지 결정합니다.
4. 계층 구조의 원하는 레벨에서 * Actions * 열의 드롭다운 목록을 선택하고 * Manage * 를 선택합니다.



5. 앱을 관리하지 않으려면 원하는 앱의 * Actions * 열에서 드롭다운 목록을 선택하고 * Ignore * 를 선택합니다.

   예를 들어, "Jenkins" 네임스페이스의 모든 앱을 함께 관리하여 동일한 스냅샷 및 백업 정책을 가지려면 네임스페이스를 관리하고 네임스페이스의 앱을 무시해야 합니다.



관리하기로 선택한 앱은 이제 * Managed * 탭에서 사용할 수 있습니다. 무시된 앱은 * ignored * 탭으로 이동합니다. 검색된 탭에 앱이 표시되지 않으므로 새 앱을 설치하면 찾아서 관리하기가 더 쉬워집니다.

Kubernetes 레이블로 앱 관리

Astra Control에는 응용 프로그램 페이지 상단에 * 사용자 정의 앱 정의 * 라는 작업이 포함되어 있습니다. 이 작업을 통해 Kubernetes 레이블로 식별된 앱을 관리할 수 있습니다. "Kubernetes 레이블로 앱 정의에 대해 자세히 알아보십시오".

단계

1. 응용 프로그램 > 사용자 정의 응용 프로그램 정의 * 를 선택합니다.
2. 사용자 정의 응용 프로그램 정의 * 대화 상자에서 응용 프로그램을 관리하는 데 필요한 정보를 제공합니다.

    a. * 새 앱 *: 앱의 표시 이름을 입력합니다.

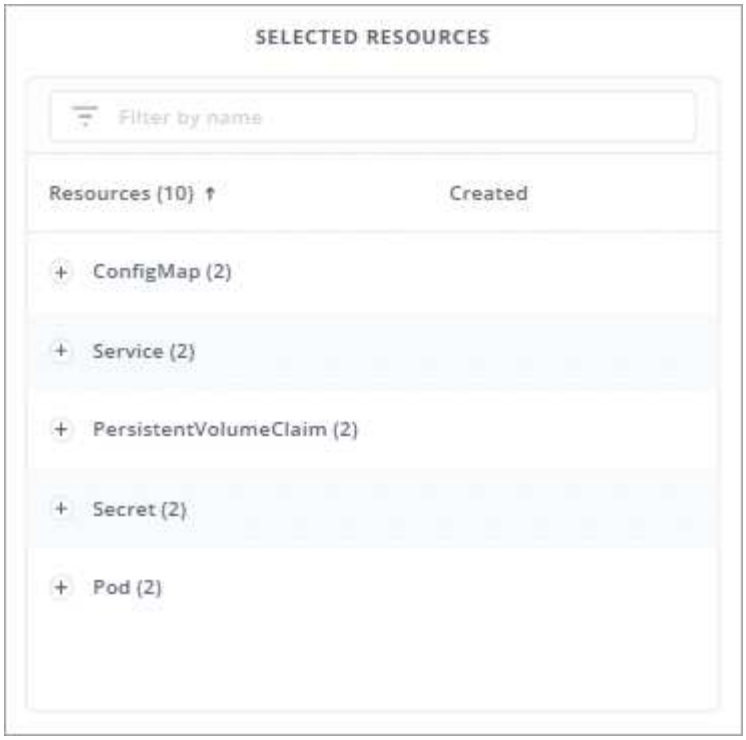
    b. * 클러스터 *: 앱이 있는 클러스터를 선택합니다.

    c. * 네임스페이스: * 앱의 네임스페이스를 선택합니다.

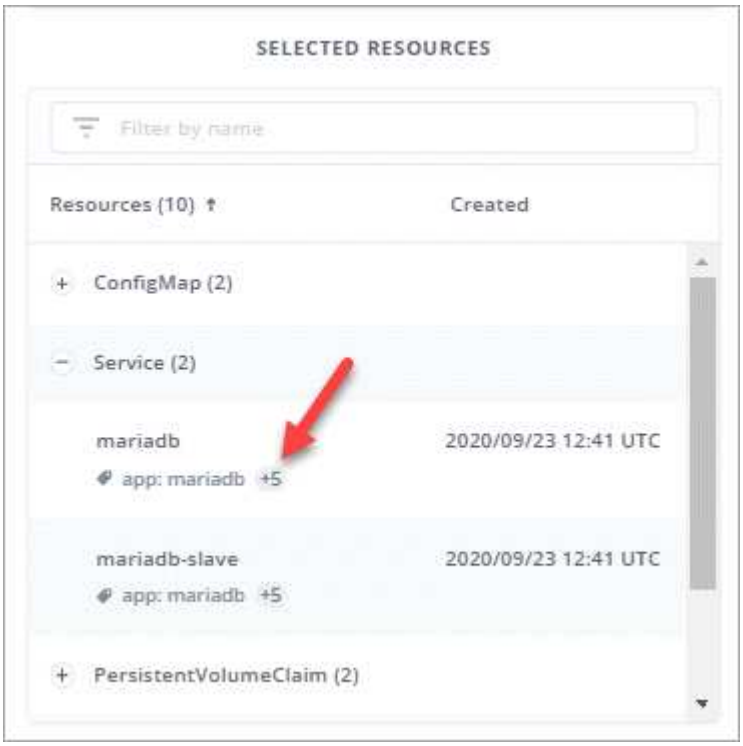
    d. * 레이블: * 레이블을 입력하거나 아래 리소스에서 레이블을 선택합니다.

    e. * 선택한 리소스 *: 보호하려는 선택한 Kubernetes 리소스(Pod, 기밀, 영구 볼륨 등)를 보고 관리합니다.

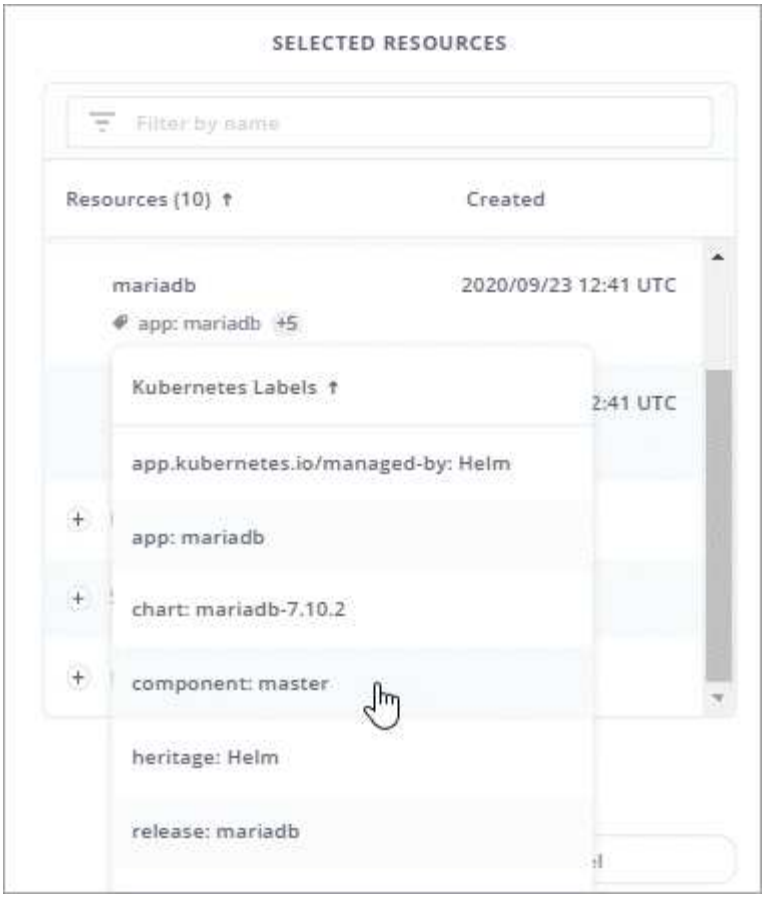
    예를 들면 다음과 같습니다.



- 리소스를 확장하고 레이블 수를 선택하여 사용 가능한 레이블을 봅니다.

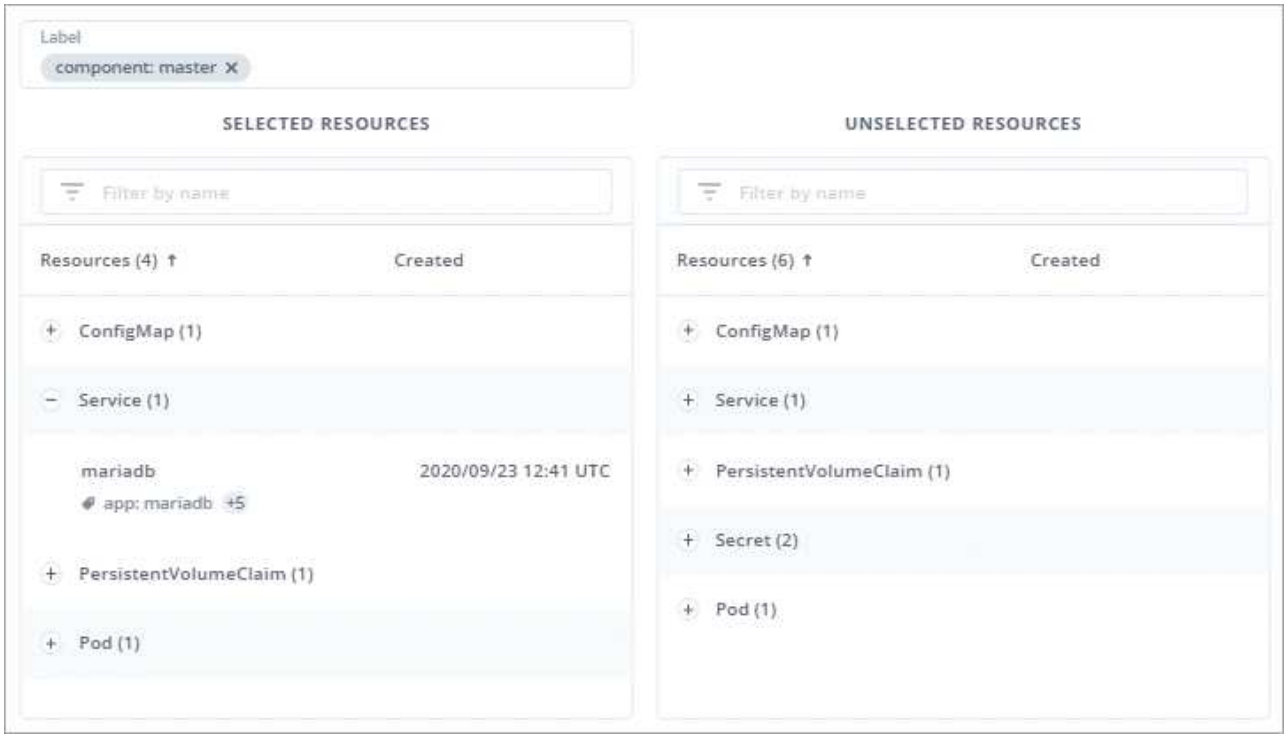


- 레이블 중 하나를 선택합니다.



레이블을 선택하면 * Label * (레이블 *) 필드에 표시됩니다. 또한 Astra Control은 선택한 레이블과 일치하지 않는 리소스를 표시하도록 * 선택되지 않은 리소스 * 섹션을 업데이트합니다.

a. 선택하지 않은 리소스 *: 보호하지 않을 앱 리소스를 확인합니다.

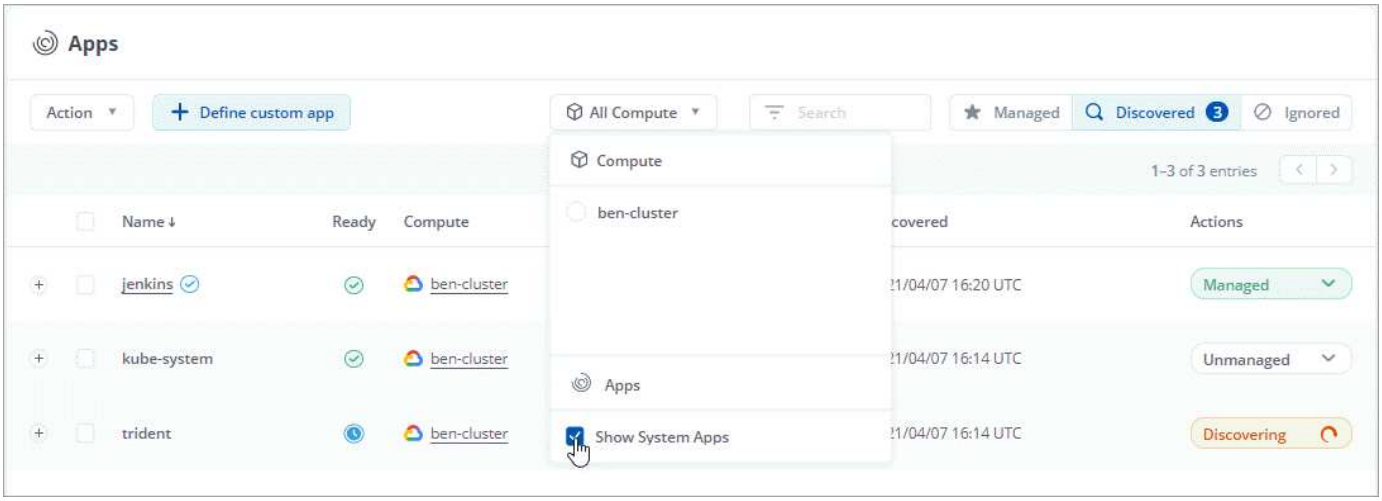


3. 사용자 지정 앱 정의 * 를 선택합니다.

Astra Control은 앱 관리를 지원합니다. 이제 * Managed * 탭에서 찾을 수 있습니다.

시스템 앱은 어떻습니까?

Astra Control은 Kubernetes 클러스터에서 실행 중인 시스템 앱을 검색합니다. 앱 목록을 필터링하여 볼 수 있습니다.



이러한 시스템 앱은 기본적으로 표시되지 않습니다. 백업해야 하는 경우는 드뭅니다.

스냅샷 및 백업으로 애플리케이션 보호

자동화된 보호 정책을 사용하거나 필요에 따라 스냅샷과 백업을 생성하여 앱을 보호합니다.

스냅샷 및 백업

snapshot_은 앱과 동일한 프로비저닝된 볼륨에 저장된 앱의 시점 복사본입니다. 대개 빠릅니다. 로컬 스냅샷은 애플리케이션을 이전 시점으로 복구하는 데 사용됩니다.

백업 _ 은(는) 클라우드의 오브젝트 스토리지에 저장됩니다. 로컬 스냅샷에 비해 백업 속도가 느려질 수 있습니다. 하지만 클라우드 내 여러 지역에서 액세스하여 앱을 마이그레이션할 수 있습니다. 또한 백업의 보존 기간을 더 길게 선택할 수도 있습니다.

> _최근 백업 _ 이(가) 있을 때까지 완전히 보호할 수 없습니다. 백업은 영구 볼륨으로부터 멀리 떨어진 개체 저장소에 저장되기 때문에 이 작업이 중요합니다. 장애 또는 사고로 인해 클러스터와 영구 스토리지가 손상된 경우 복구할 백업이 필요합니다. 스냅샷을 사용하면 복구할 수 없습니다.

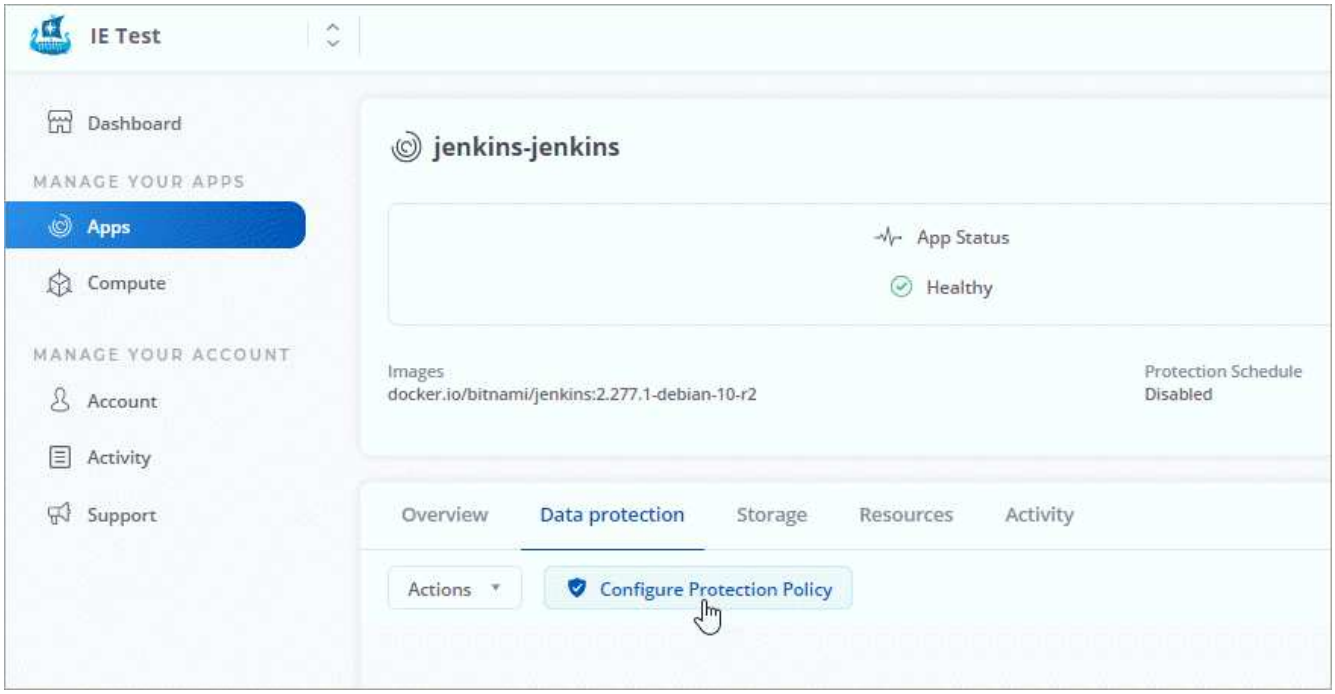
보호 정책을 구성합니다

보호 정책은 정의된 일정에 따라 스냅샷, 백업 또는 둘 다를 생성하여 앱을 보호합니다. 시간별, 일별, 주별 및 월별 스냅샷과 백업을 생성하도록 선택할 수 있으며, 보존할 복제본 수를 지정할 수 있습니다. 시간당 한 번 이상 백업 또는 스냅샷을 자주 실행해야 하는 경우 를 수행할 수 있습니다 "Astra Control REST API를 사용하여 스냅샷과 백업을 생성합니다".

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.

2. 데이터 보호 * 를 선택합니다.
3. 보호 정책 구성 * 을 선택합니다.

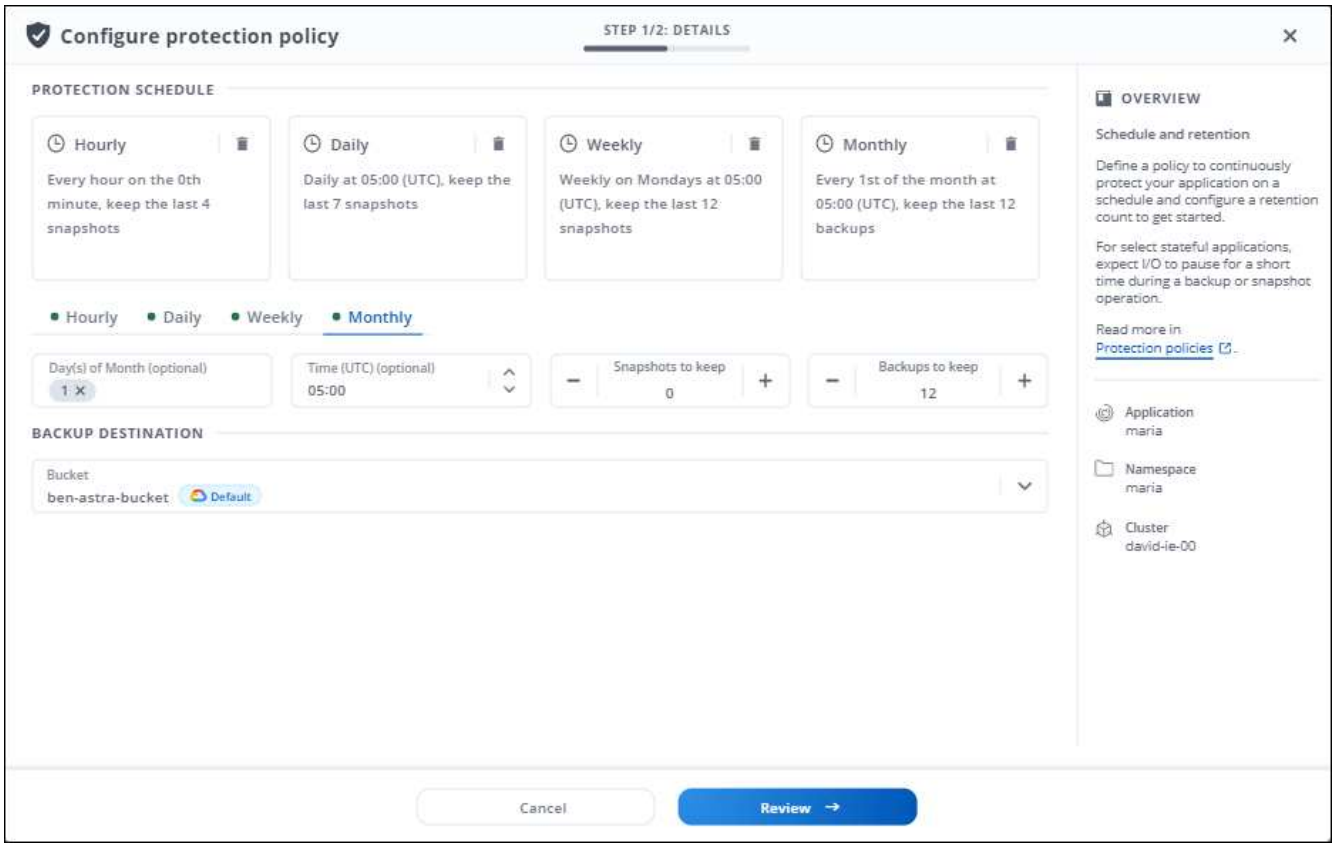


4. 시간별, 일별, 주별 및 월별 스케줄에 대해 유지할 스냅샷 및 백업 수를 선택하여 보호 스케줄을 정의합니다.

   시간별, 일별, 주별 및 월별 스케줄을 동시에 정의할 수 있습니다. 스냅샷 및 백업에 대한 보존 레벨을 설정하기 전에는 스케줄이 활성화되지 않습니다.

   백업의 보존 레벨을 설정할 때 백업을 저장할 버킷을 선택할 수 있습니다.

   다음 예에서는 스냅샷 및 백업의 경우 매시간, 일별, 주별 및 월별로 4개의 보호 스케줄을 설정합니다.



5. Review * 를 선택합니다.
6. 구성 * 을 선택합니다.

   다음은 이러한 각 단계를 보여 주는 비디오입니다.

   ► https://docs.netapp.com/ko-kr/astra-control-service/media/use/video-set-protection-policy.mp4 *(video)*

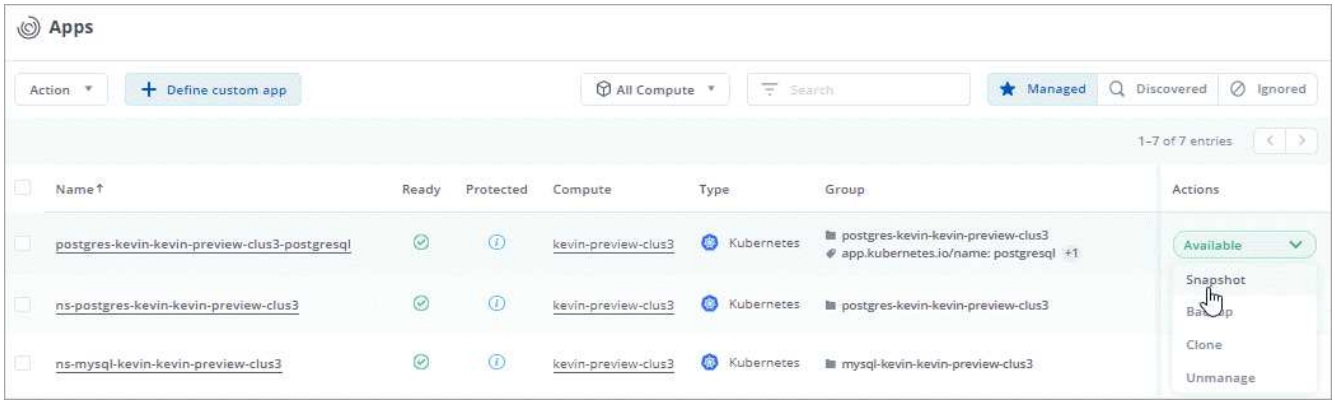
Astra Control은 정의한 스케줄 및 보존 정책을 사용하여 스냅샷 및 백업을 생성하고 유지함으로써 데이터 보호 정책을 구현합니다.

스냅샷을 생성합니다

언제든지 주문형 스냅샷을 생성할 수 있습니다.

단계

1. 응용 프로그램 * 을 선택합니다.
2. 원하는 앱의 * Actions * 열에서 드롭다운 목록을 선택합니다.
3. 스냅샷 * 을 선택합니다.



4. 스냅샷 이름을 사용자 지정한 다음 * Review Information * (검토 정보 *)을 선택합니다.
5. 스냅샷 요약을 검토하고 * Snapshot App * 을 선택합니다.

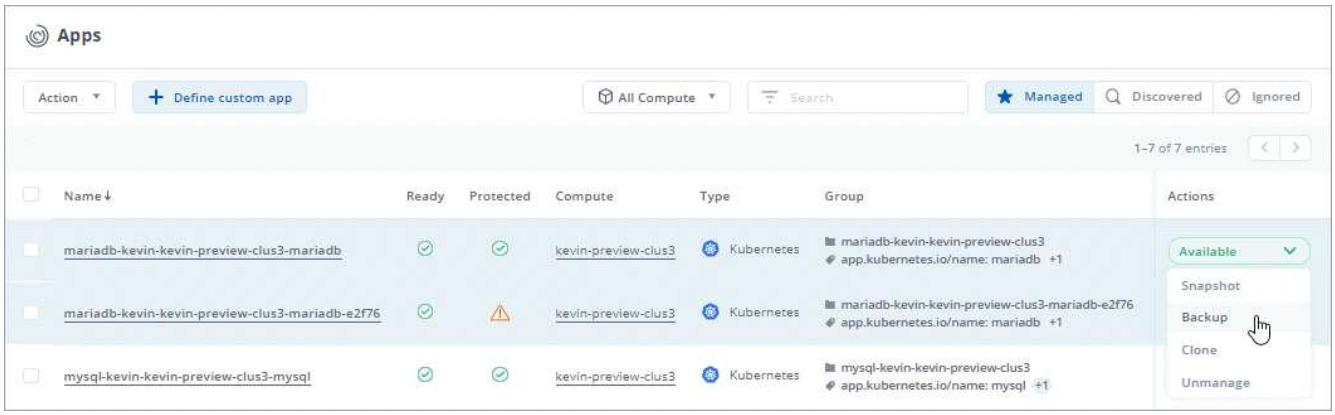
Astra Control은 앱의 스냅샷을 생성합니다.

백업을 생성합니다

언제든지 앱을 백업할 수도 있습니다.

단계

1. 응용 프로그램 * 을 선택합니다.
2. 원하는 앱의 * Actions * 열에서 드롭다운 목록을 선택합니다.
3. 백업 * 을 선택합니다.



4. 백업 이름을 사용자 지정하고 기존 스냅샷에서 앱을 백업할지 여부를 선택한 다음 * 검토 정보 * 를 선택합니다.
5. 백업 요약을 검토하고 * Backup App * 을 선택합니다.

Astra Control은 앱 백업을 생성합니다.

스냅샷 및 백업을 봅니다

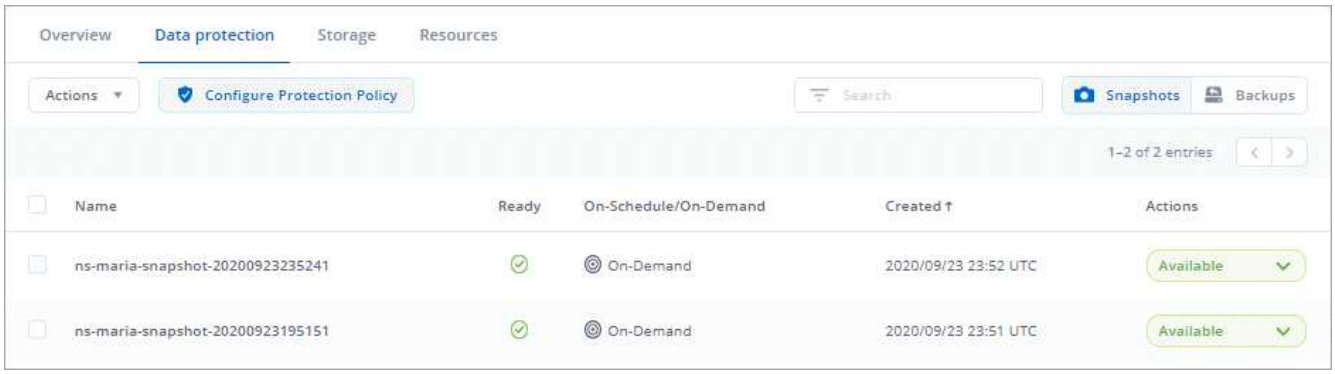
Data Protection 탭에서 앱의 스냅샷 및 백업을 볼 수 있습니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.

2. 데이터 보호 * 를 선택합니다.

   스냅샷은 기본적으로 표시됩니다.



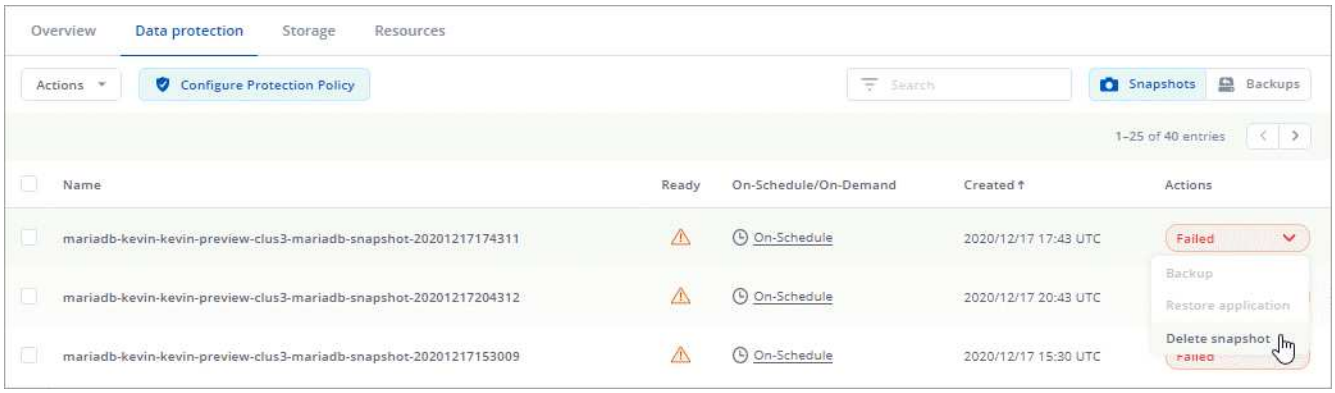
3. 백업 목록을 보려면 * backups * 를 선택합니다.

스냅샷을 삭제합니다

더 이상 필요하지 않은 예약된 스냅샷 또는 주문형 스냅샷을 삭제합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. 데이터 보호 * 를 선택합니다.
3. 원하는 스냅샷의 * Actions * 열에서 드롭다운 목록을 선택합니다.
4. 스냅샷 삭제 * 를 선택합니다.



5. 삭제를 확인할 스냅샷의 이름을 입력한 다음 * 예, 스냅샷 삭제 * 를 선택합니다.

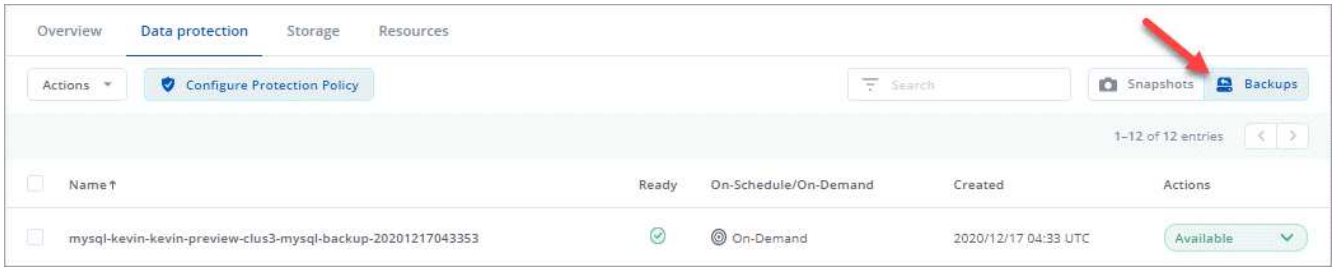
Astra Control이 스냅샷을 삭제합니다.

백업을 삭제합니다

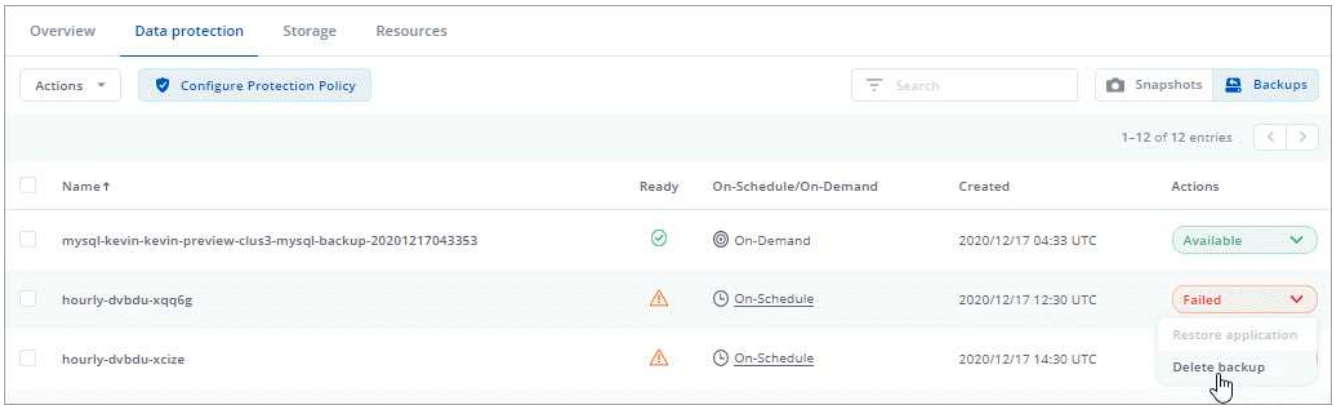
더 이상 필요하지 않은 예약된 백업 또는 필요 시 백업을 삭제합니다.

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. 데이터 보호 * 를 선택합니다.

3. Backups * 를 선택합니다.



4. 원하는 백업의 * Actions * 열에서 드롭다운 목록을 선택합니다.
5. 백업 삭제 * 를 선택합니다.



6. 삭제를 확인할 백업 이름을 입력한 다음 * 예, 백업 삭제 * 를 선택합니다.

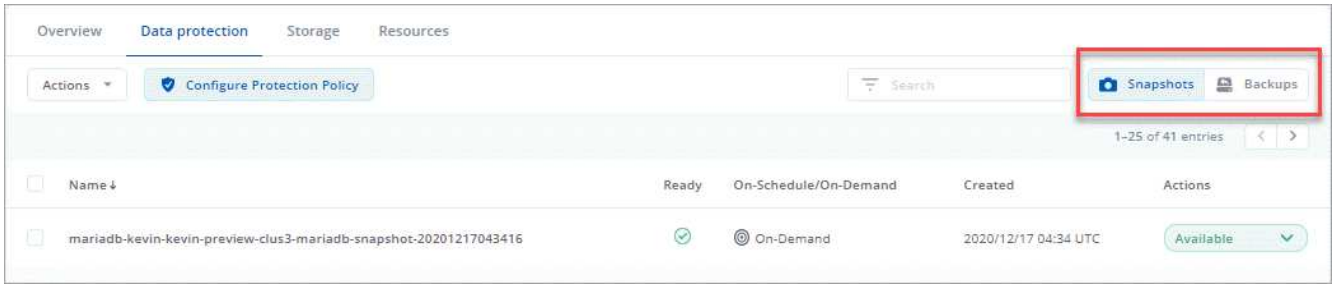
Astra Control이 백업을 삭제합니다.

앱 복원

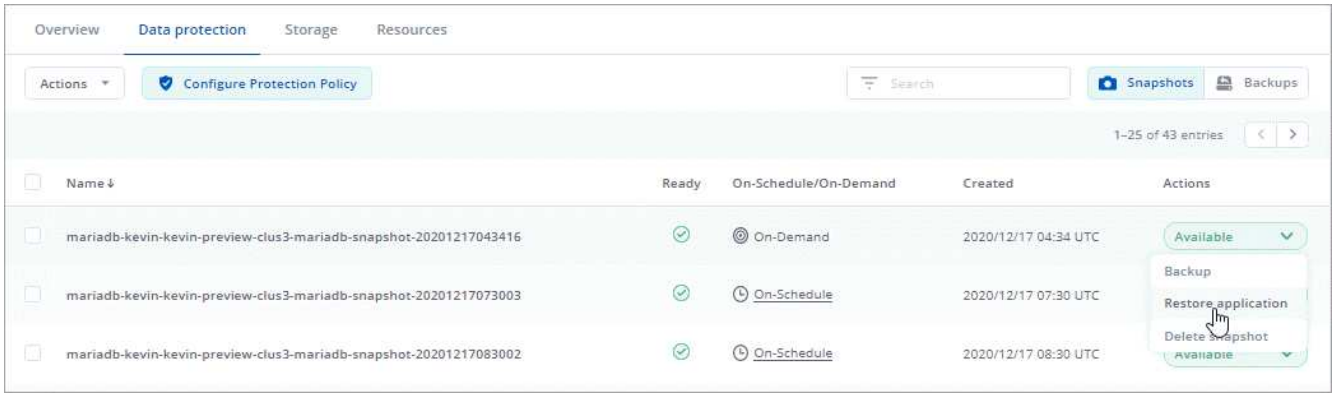
Astra Control은 스냅샷 또는 백업에서 애플리케이션을 복원할 수 있습니다. 애플리케이션을 동일한 클러스터로 복구할 경우 기존 스냅샷에서 복구하는 속도가 빨라집니다. Astra Control UI 또는 를 사용할 수 있습니다 "Astra Control API" 앱을 복원합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. 데이터 보호 * 를 선택합니다.
3. 스냅샷에서 복구하려면 * Snapshots * 를 선택한 상태로 유지합니다. 그렇지 않으면 * backups * 를 선택하여 백업에서 복원합니다.



4. 복구할 스냅샷 또는 백업의 * Actions * 열에서 드롭다운 목록을 선택합니다.
5. Restore application * 을 선택합니다.



6. * Restore details *: 복원된 앱에 대한 세부 정보를 지정합니다. 현재 실행 중인 동일한 네임스페이스 및 대상 클러스터를 선택하여 앱을 원래 상태로 복원합니다(앱을 이전 버전으로 되돌리기).
   - 앱의 네임스페이스를 입력합니다.
   - 앱의 대상 클러스터를 선택합니다.
   - 검토 * 를 클릭합니다.
7. * 복원 요약 *: 복원 작업에 대한 세부 정보를 검토하고 "복원"을 입력한 다음 * 복원 * 을 선택합니다.

Astra Control은 사용자가 제공한 정보를 기반으로 앱을 복원합니다. 앱을 제자리에 복원한 경우 연결된 영구 볼륨이 삭제되고 다시 생성됩니다.

애플리케이션 클론 복제 및 마이그레이션

기존 앱을 클론 복제하여 동일한 Kubernetes 클러스터 또는 다른 클러스터에 중복 앱을 생성합니다. Kubernetes 클러스터 간에 애플리케이션 및 스토리지를 이동해야 하는 경우 클로닝에 도움이 될 수 있습니다. 예를 들어, CI/CD 파이프라인과 Kubernetes 네임스페이스 전체에서 워크로드를 이동할 수 있습니다.

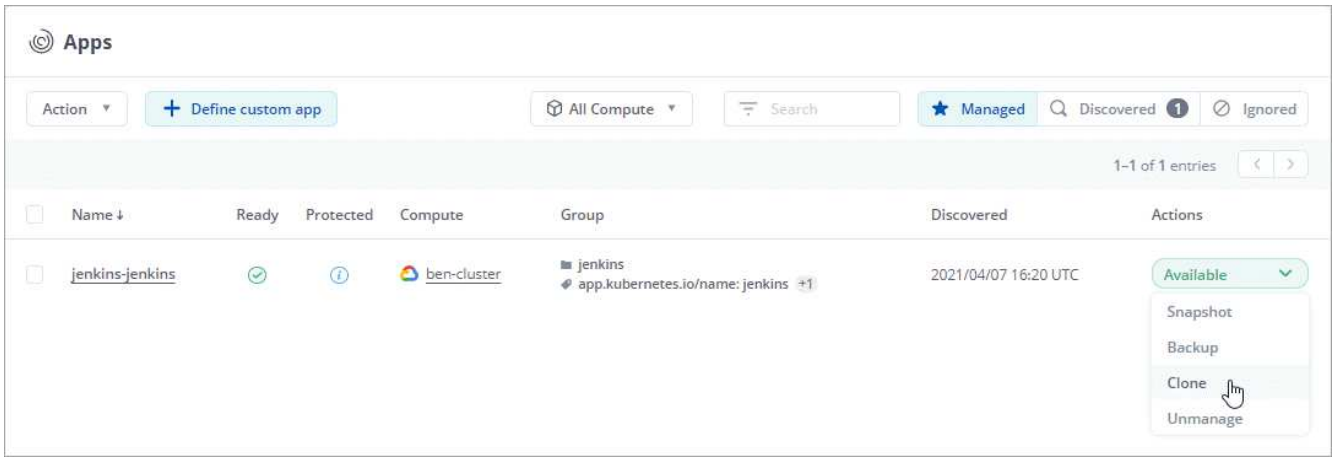
Astra Control은 앱을 클론할 때 애플리케이션 구성 및 영구 스토리지의 클론을 생성합니다.

> Jenkins CI의 운영자 배포 인스턴스를 복제하는 경우 영구 데이터를 수동으로 복원해야 합니다. 이는 앱 배포 모델의 제한 사항입니다.

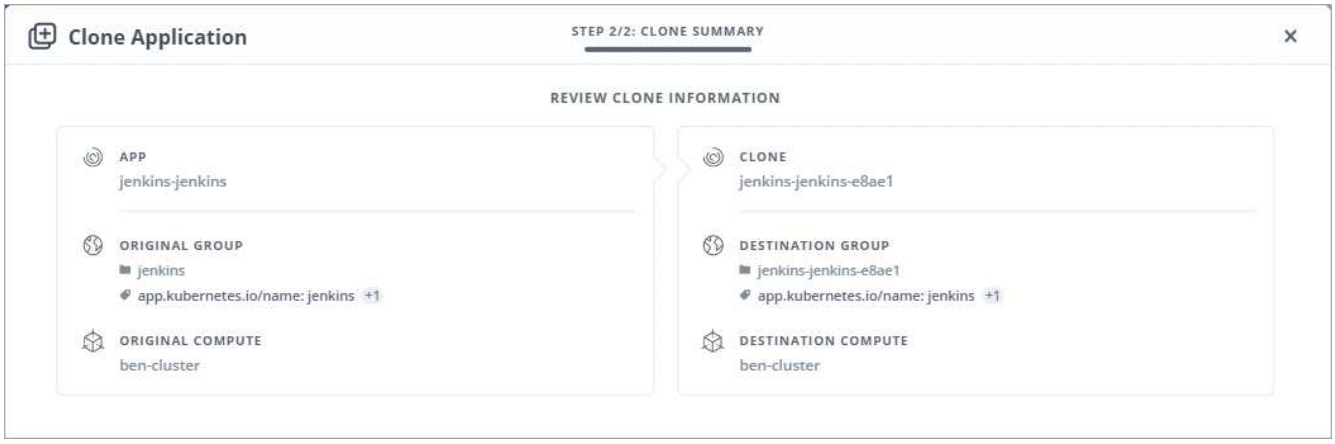
> StorageClass가 명시적으로 설정된 앱을 배포하고 앱을 복제해야 하는 경우 타겟 클러스터에 원래 지정된 StorageClass가 있어야 합니다. 명시적으로 StorageClass를 동일한 StorageClass가 없는 클러스터로 설정한 애플리케이션을 클론 복제하면 실패합니다.

단계

1. 응용 프로그램 * 을 선택합니다.
2. 원하는 앱의 * Action * 열에서 드롭다운 목록을 선택합니다.
3. 클론 * 을 선택합니다.



4. * 클론 세부 정보 *: 클론에 대한 세부 정보 지정:
   - 기본 이름과 네임스페이스를 유지하거나 편집합니다.
   - 클론의 대상 클러스터를 선택합니다.
   - 기존 스냅샷이나 백업에서 클론을 생성할지 여부를 선택합니다. 이 옵션을 선택하지 않으면 Astra Control이 앱의 현재 상태에서 클론을 생성합니다.
5. * 클론 요약 *: 클론에 대한 세부 정보를 검토하고 * Clone App * 을 선택합니다.



Astra Control은 사용자가 제공한 정보를 기반으로 해당 앱을 복제합니다.

앱 실행 후크 관리

실행 후크는 관리되는 앱의 스냅샷 전후에 실행할 수 있는 사용자 지정 스크립트입니다. 예를 들어 데이터베이스 앱이 있는 경우 실행 후크를 사용하여 스냅샷 전에 모든 데이터베이스 트랜잭션을 일시 중지하고 스냅샷이 완료된 후 트랜잭션을 다시 시작할 수 있습니다. 따라서 애플리케이션 정합성이 보장되는 스냅샷이 보장됩니다.

기본 실행 후크 및 정규식

일부 애플리케이션의 경우 Astra Control은 NetApp에서 제공하는 기본 실행 후크와 함께 제공되며, 스냅샷 전후에 고정 및 고정 작업을 처리합니다. Astra Control은 정규식을 사용하여 앱의 컨테이너 이미지를 다음과 같은 앱에 일치시킵니다.

- MariaDB
  - 일치하는 정규식:\bmariadb\b
- MySQL
  - 일치 정규식:\bmysql\b
- PostgreSQL
  - 일치하는 정규식:\bpostgresql\b

일치하는 항목이 있으면 해당 앱에 대한 NetApp 제공 기본 실행 후크가 앱의 활성 실행 후크 목록에 나타나고, 해당 앱의 스냅샷을 생성하면 해당 후크가 자동으로 실행됩니다. 사용자 지정 앱 중 하나에 정규식과 일치하는 유사한 이미지 이름이 있는 경우(기본 실행 후크를 사용하지 않으려는 경우) 이미지 이름을 변경할 수 있습니다. 또는 해당 앱에 대한 기본 실행 후크를 비활성화하고 대신 사용자 지정 후크를 사용합니다.

기본 실행 후크는 삭제하거나 수정할 수 없습니다.

사용자 정의 실행 후크에 대한 중요 참고 사항

앱에 대한 실행 후크를 계획할 때 다음 사항을 고려하십시오.

- Astra Control을 사용하려면 실행 가능한 셸 스크립트 형식으로 실행 후크를 작성해야 합니다.
- 스크립트 크기는 128KB로 제한됩니다.
- Astra Control은 실행 후크 설정 및 모든 일치 기준을 사용하여 스냅샷에 적용할 후크를 결정합니다.
- 모든 실행 후크 오류는 소프트 장애이며, 후크에 장애가 발생해도 다른 후크와 스냅샷이 시도됩니다. 그러나 후크가 실패하면 * Activity * 페이지 이벤트 로그에 경고 이벤트가 기록됩니다.
- 실행 후크를 생성, 편집 또는 삭제하려면 소유자, 관리자 또는 구성원 권한이 있는 사용자여야 합니다.
- 실행 후크를 실행하는 데 25분 이상 걸리는 경우 후크에 장애가 발생하고 반환 코드가 "N/A"인 이벤트 로그 항목이 생성됩니다. 영향을 받는 모든 스냅샷은 시간 초과되어 실패로 표시되며, 그 결과 이벤트 로그 항목이 시간 초과를 나타냅니다.

> 실행 후크는 실행 중인 응용 프로그램의 기능을 줄이거나 완전히 비활성화하기 때문에 사용자 지정 실행 후크가 실행되는 시간을 최소화해야 합니다.

스냅샷이 실행되면 실행 후크 이벤트가 다음 순서로 발생합니다.

1. NetApp에서 제공하는 기본 사전 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다.
2. 해당되는 모든 사용자 지정 사전 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다. 필요에 따라 사용자 지정 사전 스냅샷 후크를 생성하고 실행할 수 있지만 스냅샷이 보장되거나 구성 가능해지기 전에 이러한 후크의 실행 순서가 보장되지 않습니다.
3. 스냅샷이 수행됩니다.
4. 해당되는 모든 사용자 지정 사후 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다. 필요에 따라 사용자 지정 사후 스냅샷 후크를 생성하고 실행할 수 있지만 스냅샷 후에 이러한 후크를 실행하는 순서는 보장되거나 구성할 수 없습니다.
5. NetApp에서 제공하는 모든 기본 사후 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다.

> 운영 환경에서 실행 후크 스크립트를 사용하려면 항상 해당 스크립트를 테스트해야 합니다. 'kubbeck exec' 명령을 사용하여 스크립트를 편리하게 테스트할 수 있습니다. 운영 환경에서 실행 후크를 활성화한 후 결과 스냅샷을 테스트하여 정합성이 보장되는지 확인합니다. 앱을 임시 네임스페이스에 클론 복제하고 스냅샷을 복구한 다음 앱을 테스트하여 이 작업을 수행할 수 있습니다.

기존 실행 후크를 봅니다

앱에 대한 기존 맞춤형 또는 NetApp에서 제공한 기본 실행 후크를 볼 수 있습니다.

단계

1. 응용 프로그램 * 으로 이동한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.

   결과 목록에서 사용 가능하거나 비활성화된 실행 후크를 모두 볼 수 있습니다. 후크의 상태, 소스 및 실행 시간(사전 또는 사후 스냅샷)을 확인할 수 있습니다. 실행 후크를 둘러싼 이벤트 로그를 보려면 왼쪽 탐색 영역의 * Activity * 페이지로 이동합니다.

사용자 지정 실행 후크를 만듭니다

앱의 사용자 정의 실행 후크를 만들 수 있습니다. 을 참조하십시오 "실행 후크 예" 후크 예 실행 후크를 만들려면 소유자, 관리자 또는 구성원 권한이 있어야 합니다.

> 실행 후크로 사용할 사용자 정의 쉘 스크립트를 작성할 때는 Linux 명령을 실행하거나 실행 파일에 대한 전체 경로를 제공하지 않는 한 파일 시작 부분에 적절한 셸을 지정해야 합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.
3. 새 후크 추가 * 를 선택합니다.
4. 후크 세부 정보 * 영역에서 후크를 실행해야 하는 시기에 따라 * 사전 스냅샷 * 또는 * 사후 스냅샷 * 을 선택합니다.
5. 후크의 고유한 이름을 입력합니다.
6. (선택 사항) 실행 중에 후크에 전달할 인수를 입력하고 각 인수 뒤에 Enter 키를 눌러 각 인수를 기록합니다.
7. Container Images * (컨테이너 이미지 *) 영역에서 응용 프로그램에 포함된 모든 컨테이너 이미지에 대해 후크를 실행해야 하는 경우 * Apply to all container images * (모든 컨테이너 이미지에 적용) 확인란을 활성화합니다. 대신 후크가 하나 이상의 지정된 컨테이너 이미지에만 동작해야 하는 경우 일치시킬 * 컨테이너 이미지 이름 필드에 컨테이너 이미지 이름을 입력합니다.
8. Script * 영역에서 다음 중 하나를 수행합니다.
   - 사용자 지정 스크립트를 업로드합니다.
     i. 파일 업로드 * 옵션을 선택합니다.
     ii. 파일을 찾아 업로드합니다.
     iii. 스크립트에 고유한 이름을 지정합니다.
     iv. (선택 사항) 다른 관리자가 스크립트에 대해 알아야 하는 참고 사항을 입력합니다.
   - 클립보드에서 사용자 정의 스크립트를 붙여 넣습니다.

    i. 클립보드에서 붙여넣기 * 옵션을 선택합니다.
    ii. 텍스트 필드를 선택하고 필드에 스크립트 텍스트를 붙여 넣습니다.
    iii. 스크립트에 고유한 이름을 지정합니다.
    iv. (선택 사항) 다른 관리자가 스크립트에 대해 알아야 하는 참고 사항을 입력합니다.
9. 후크 추가 * 를 선택합니다.

실행 후크를 비활성화합니다

앱 스냅샷 전후에 실행 후크가 실행되지 않도록 임시로 설정하려면 실행 후크를 사용하지 않도록 설정할 수 있습니다. 실행 후크를 비활성화하려면 소유자, 관리자 또는 구성원 권한이 있어야 합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.
3. 비활성화하려는 후크에 대해 * 작업 * 드롭다운을 선택합니다.
4. 비활성화 * 를 선택합니다.

실행 후크를 삭제합니다

더 이상 필요 없는 경우 실행 후크를 완전히 제거할 수 있습니다. 실행 후크를 삭제하려면 소유자, 관리자 또는 구성원 권한이 있어야 합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.
3. 삭제할 후크에 대한 * 작업 * 드롭다운을 선택합니다.
4. 삭제 * 를 선택합니다.

실행 후크 예

다음 예제를 사용하여 실행 후크를 구조화하는 방법에 대해 알아보십시오. 이러한 후크를 템플릿 또는 테스트 스크립트로 사용할 수 있습니다.

간단한 성공 사례

다음은 표준 출력 및 표준 오류에 성공하여 메시지를 기록하는 간단한 후크의 예입니다.

```
#!/bin/sh

# success_sample.sh
#
# A simple noop success hook script for testing purposes.
#
# args: None
```

```
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.sh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

단순한 성공 사례**(bash** 버전**)**

다음은 bash용으로 작성된 표준 출력 및 표준 오류에 성공하여 메시지를 쓰는 간단한 후크의 예입니다.

```
#!/bin/bash

# success_sample.bash
#
# A simple noop success hook script for testing purposes.
#
# args: None


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.bash"

# exit with 0 to indicate success
```

```
info "exit 0"
exit 0
```

간단한 성공 사례(**zsh** 버전)

다음은 Z 셸에 대해 작성된 표준 출력 및 표준 오류에 성공하여 메시지를 기록하는 간단한 후크의 예입니다.

```
#!/bin/zsh

# success_sample.zsh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}

#
```

```
# main
#

# log something to stdout
info "running success_sample.zsh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

인수 성공 예제

다음 예제에서는 후크에 args를 사용하는 방법을 보여 줍니다.

```
#!/bin/sh

# success_sample_args.sh
#
# A simple success hook script with args for testing purposes.
#
# args: Up to two optional args that are echoed to stdout

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
```

```
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample_args.sh"


# collect args
arg1=$1
arg2=$2

# output args and arg count to stdout
info "number of args: $#"
info "arg1 ${arg1}"
info "arg2 ${arg2}"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

사전 스냅샷/사후 스냅샷 후크의 예

다음 예제에서는 사전 스냅샷 및 사후 스냅샷 후크에 대해 동일한 스크립트를 사용하는 방법을 보여 줍니다.

```
#!/bin/sh

# success_sample_pre_post.sh
#
# A simple success hook script example with an arg for testing purposes
# to demonstrate how the same script can be used for both a prehook and
posthook
#
# args: [pre|post]

# unique error codes for every error case
ebase=100
eusage=$((ebase+1))
ebadstage=$((ebase+2))
epre=$((ebase+3))
epost=$((ebase+4))
```

```
#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# Would run prehook steps here
#
prehook() {
    info "Running noop prehook"
    return 0
}

#
# Would run posthook steps here
#
posthook() {
    info "Running noop posthook"
    return 0
}
```

```
#
# main
#

# check arg
stage=$1
if [ -z "${stage}" ]; then
    echo "Usage: $0 <pre|post>"
    exit ${eusage}
fi

if [ "${stage}" != "pre" ] && [ "${stage}" != "post" ]; then
    echo "Invalid arg: ${stage}"
    exit ${ebadstage}
fi

# log something to stdout
info "running success_sample_pre_post.sh"

if [ "${stage}" = "pre" ]; then
    prehook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during prehook"
    fi
fi

if [ "${stage}" = "post" ]; then
    posthook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during posthook"
    fi
fi

exit ${rc}
```

실패 예

다음 예제에서는 후크의 장애를 처리하는 방법을 보여 줍니다.

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
```

```
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}



#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"
```

```
# exit with specified exit code
exit ${argexitcode}
```

자세한 정보 표시 실패 예

다음 예제에서는 더 자세한 정보 로깅을 사용하여 후크의 오류를 처리하는 방법을 보여 줍니다.

```
#!/bin/sh

# failure_sample_verbose.sh
#
# A simple failure hook script with args for testing purposes.
#
# args: [The number of lines to output to stdout]


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
```

```
# main
#

# log something to stdout
info "running failure_sample_verbose.sh"


# output arg value to stdout
linecount=$1
info "line count ${linecount}"

# write out a line to stdout based on line count arg
i=1
while [ "$i" -le ${linecount} ]; do
    info "This is line ${i} from failure_sample_verbose.sh"
    i=$(( i + 1 ))
done

error "exiting with error code 8"
exit 8
```

종료 코드 예제에 오류가 발생했습니다

다음 예제에서는 종료 코드와 함께 후크 실패를 보여 줍니다.

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
```

```
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

실패 후 성공 예

다음 예제에서는 후크가 처음 실행될 때 후크가 실패하지만 두 번째 실행 후에 후크가 발생하는 방법을 보여 줍니다.

```
#!/bin/sh

# failure_then_success_sample.sh
#
# A hook script that fails on initial run but succeeds on second run for
testing purposes.
#
# Helpful for testing retry logic for post hooks.
#
```

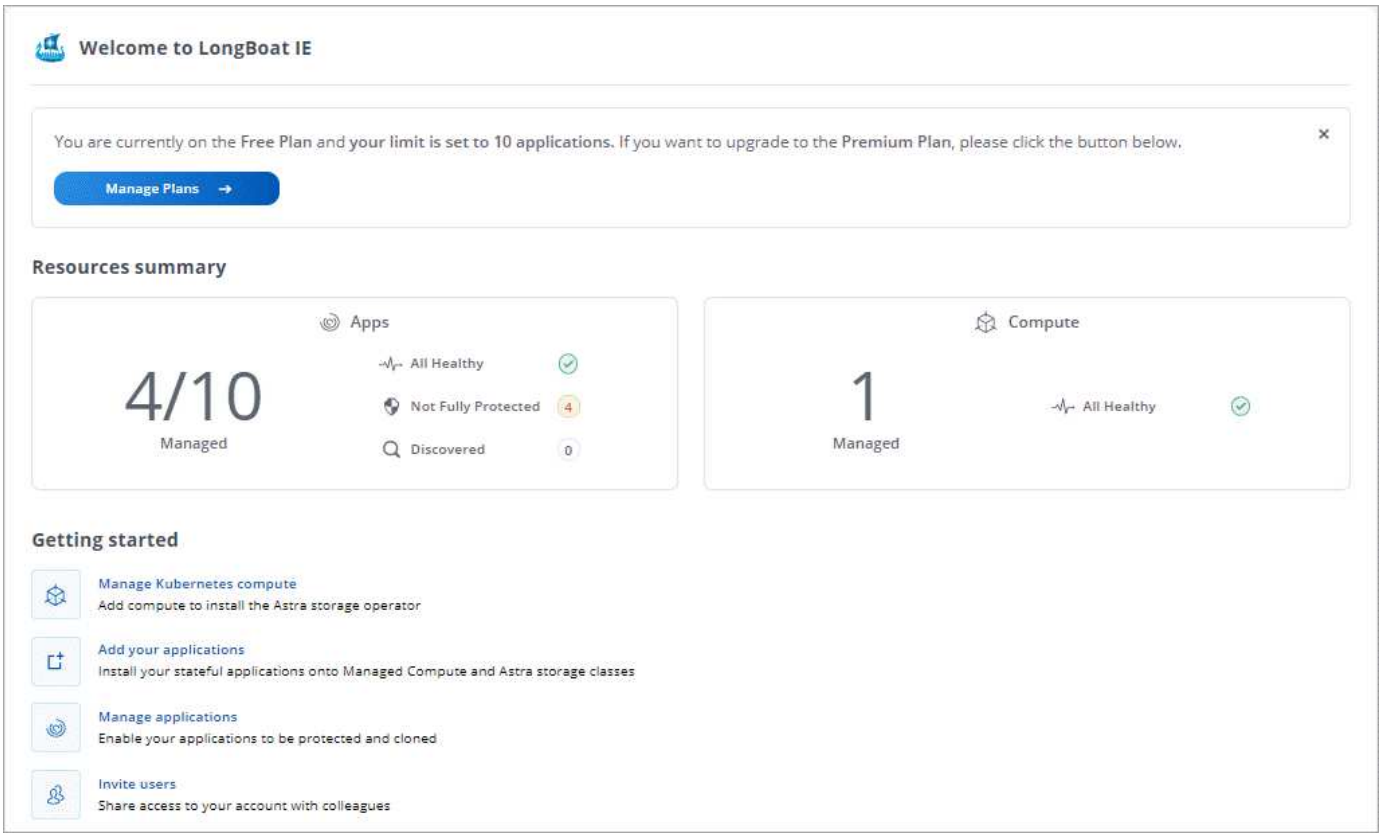
```
# args: None
#

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_success sample.sh"


if [ -e /tmp/hook-test.junk ] ; then
    info "File does exist.  Removing /tmp/hook-test.junk"
    rm /tmp/hook-test.junk
    info "Second run so returning exit code 0"
    exit 0
else
```

```
    info "File does not exist.  Creating /tmp/hook-test.junk"
    echo "test" > /tmp/hook-test.junk
    error "Failed first run, returning exit code 5"
    exit 5
fi
```

앱 및 컴퓨팅 상태 보기

앱 및 클러스터 상태 요약 보기

대시보드 * 를 클릭하여 앱, 클러스터 및 해당 상태를 한눈에 파악할 수 있습니다.



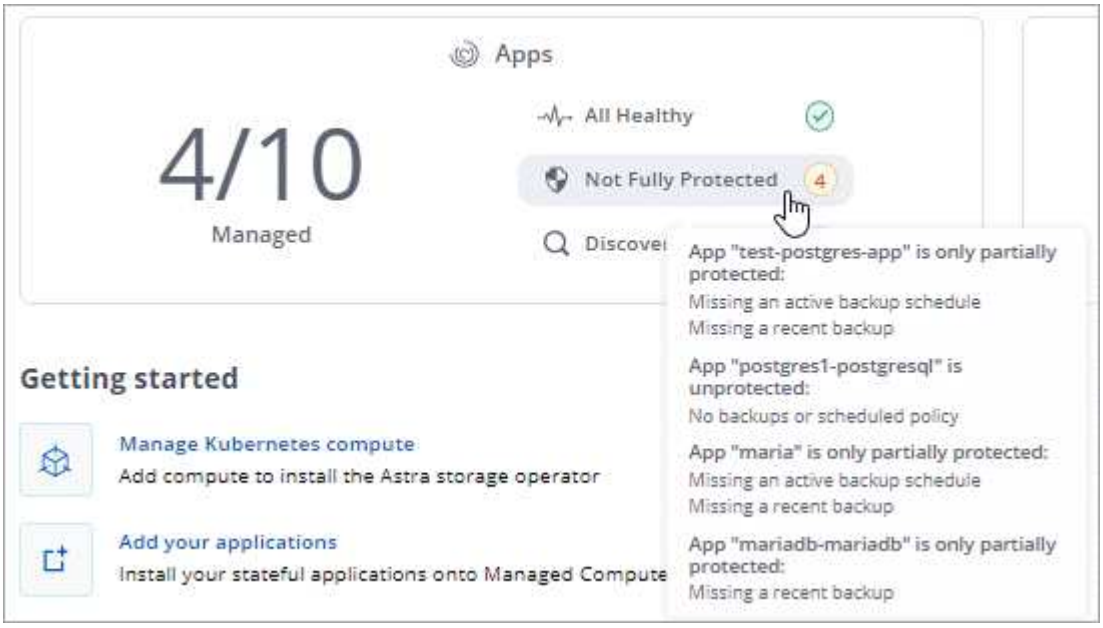
앱 타일은 다음을 식별하는 데 도움이 됩니다.

- 현재 관리하고 있는 앱 수
- 관리된 앱이 정상 상태인지 여부
- 애플리케이션이 완전히 보호되는지 여부(최근 백업을 사용할 수 있는 경우 보호됨)
- 검색되었지만 아직 관리되지 않은 앱의 수입니다.

  앱을 검색한 후 관리하거나 무시하면 되므로 이 숫자는 0이 되는 것이 좋습니다. 그런 다음 대시보드에서 검색된 앱의 수를 모니터링하여 개발자가 클러스터에 새 앱을 추가하는 시기를 파악할 수 있습니다.

이러한 항목은 단순한 숫자 또는 상태가 아니라 각 상태별로 드릴다운할 수 있습니다. 예를 들어 앱이 완전히 보호되지

않은 경우 아이콘 위로 마우스를 가져가면 완전히 보호되지 않은 앱을 확인할 수 있습니다. 여기에는 이유가 포함됩니다.



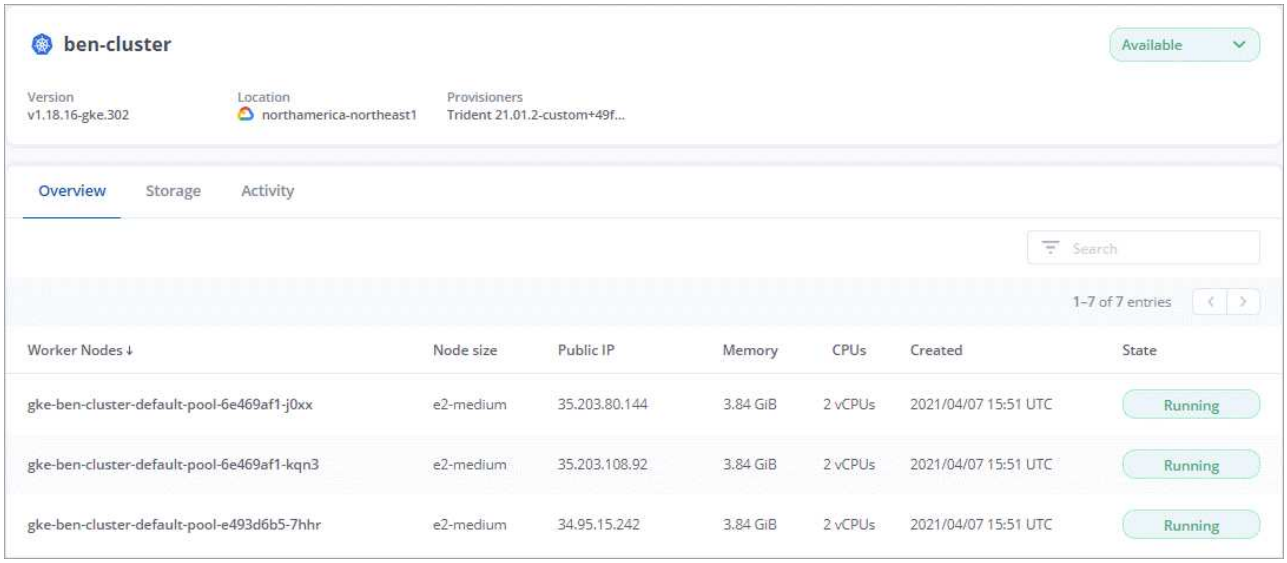
클러스터 타일은 클러스터 상태에 대한 유사한 세부 정보를 제공하며 앱을 사용하는 것처럼 드릴다운하여 자세한 정보를 얻을 수 있습니다.

클러스터의 상태 및 세부 정보를 봅니다

Astra Control에 Kubernetes 클러스터를 추가한 후에는 클러스터의 위치, 작업자 노드, 영구 볼륨 및 스토리지 클래스 등과 같은 클러스터에 대한 세부 정보를 볼 수 있습니다.

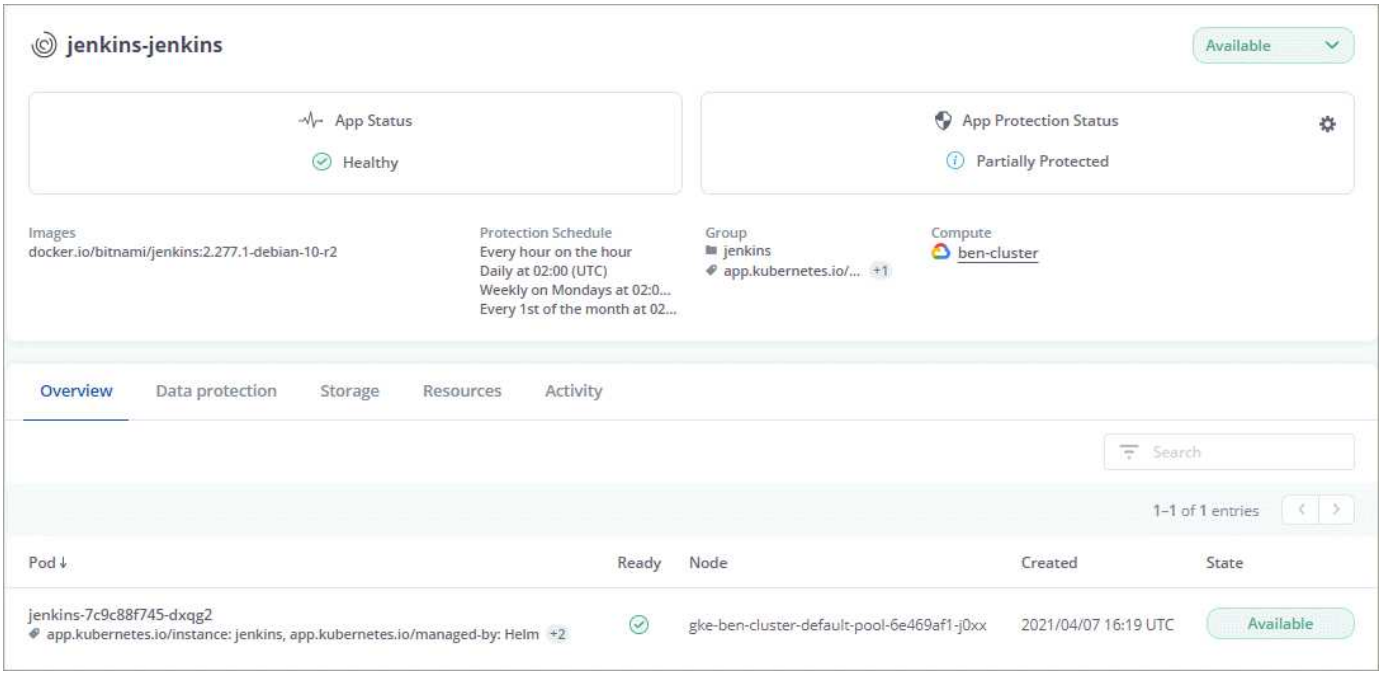
단계

1. 클러스터 * 를 선택합니다.
2. 클러스터 이름을 선택합니다.
3. Overview * 및 * Storage * 탭에서 원하는 정보를 찾아보십시오.
    - * 개요 *: 해당 상태를 포함한 작업자 노드에 대한 세부 정보.
    - * 스토리지 *: 스토리지 클래스 및 상태를 포함하여 클러스터와 연결된 영구 볼륨입니다.
    - * 활동 *: 클러스터와 관련된 Astra 활동.



앱의 상태 및 세부 정보를 봅니다

앱 관리를 시작한 후 Astra Control은 앱의 상태(정상 여부), 보호 상태(장애 시 완전히 보호되는지 여부), Pod, 영구 스토리지 등을 식별할 수 있는 앱에 대한 세부 정보를 제공합니다.



단계

1. 응용 프로그램 * 을 선택한 다음 앱 이름을 선택합니다.
2. 원하는 정보를 찾으려면 주변을 선택하십시오.

   앱 상태

   Kubernetes의 앱 상태를 반영하는 상태를 제공합니다. 예를 들어, Pod와 영구 볼륨을 온라인으로 전환합니까? 앱이 정상 상태가 아닌 경우 Kubernetes 로그를 확인하여 클러스터에서 문제를 해결해야 합니다. Astra Control은 고장 난 앱을 수정하는 데 도움이 되는 정보를 제공하지 않습니다.

앱 보호 상태

- * 완전 보호 *: 이 앱에는 활성 백업 스케줄과 1주일 미만의 성공적인 백업이 있습니다
- * 부분 보호됨 *: 응용 프로그램에 활성 백업 일정, 활성 스냅샷 일정 또는 백업 또는 스냅샷이 있습니다
- * 보호되지 않음 *: 완전히 보호되거나 부분적으로 보호되지 않는 앱

  _최근 백업 _ 이(가) 있을 때까지 완전히 보호할 수 없습니다. 백업은 영구 볼륨으로부터 멀리 떨어진 개체 저장소에 저장되기 때문에 이 작업이 중요합니다. 장애 또는 사고로 인해 클러스터가 삭제되며 영구적 저장소인 경우 복구할 백업이 필요합니다. 스냅샷을 사용하면 복구할 수 없습니다.

개요

앱과 연결된 Pod의 상태에 대한 정보입니다.

데이터 보호

데이터 보호 정책을 구성하고 기존 스냅샷 및 백업을 볼 수 있습니다.

스토리지

에는 애플리케이션 레벨의 영구 볼륨이 나와 있습니다. 영구 볼륨의 상태는 Kubernetes 클러스터의 관점에서 나옵니다.

리소스

백업 및 관리되는 리소스를 확인할 수 있습니다.

활동입니다

앱과 관련된 Astra Control 활동입니다.

버킷을 관리합니다

추가 버킷을 추가하고 클라우드 공급자의 Kubernetes 클러스터 기본 버킷을 변경하여 Astra가 백업 및 클론에 사용하는 버킷을 관리합니다.

관리자만 버킷을 추가하고 수정할 수 있습니다.

Astra Control이 버킷을 사용하는 방식

첫 번째 Kubernetes 클러스터 관리를 시작할 때 Astra Control Service는 관리형 클러스터와 같은 지리적 위치에서의 클라우드 공급자 기본 버킷을 생성합니다.

Astra Control Service는 사용자가 생성한 백업 및 클론에 이 기본 버킷을 사용합니다. 그런 다음 백업을 사용하여 클러스터 간에 앱을 복원 및 클론 복제할 수 있습니다.

Astra Control Service에 추가 버킷을 추가하는 경우 보호 정책을 생성할 때 해당 버킷 중에서 선택할 수 있습니다. 또한 Astra Control Service가 임시 백업 및 클론에 사용하는 기본 버킷을 변경할 수 있습니다. 하나의 클라우드 공급자에서 다른 클라우드 공급자로 버킷을 클론 복제할 수 없습니다. 클라우드 공급자 간에 데이터를 마이그레이션하려면 연결된 애플리케이션을 클론 복제하십시오.

Astra Control Service는 백업 또는 클론을 시작하기 전에 대상 버킷에 액세스할 수 있는지 여부를 확인합니다.

기존 버킷을 봅니다

Astra Control Service에서 사용할 수 있는 버킷 목록을 보고 해당 상태를 확인하고 클라우드 공급자의 기본 버킷을 식별합니다.

버킷은 다음 상태 중 하나를 가질 수 있습니다.

보류 중
: 버킷을 추가한 후 보류 중인 상태에서 시작하여 Astra Control이 처음으로 이 버킷을 확인합니다.

사용 가능
: 이 버킷은 Astra Control에서 사용할 수 있습니다.

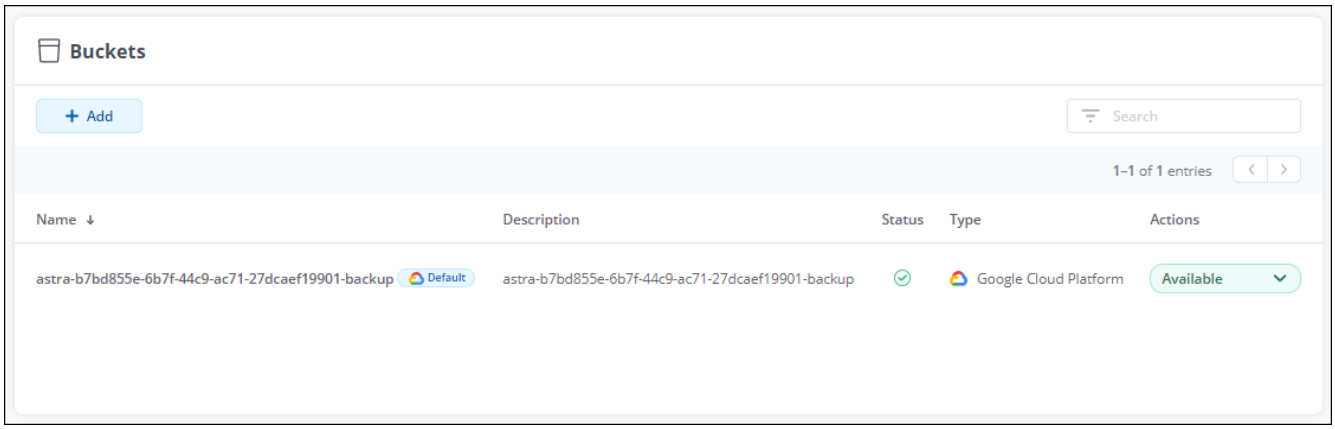
제거되었습니다
: 버킷이 제거된 상태인 경우에도 기본 버킷으로 설정하고 보호 일정에 할당할 수 있습니다. 하지만 데이터 보호 작업이 시작될 때 버킷이 가용 상태가 아니면 해당 작업이 실패합니다.

단계

1. 스토리지 관리 * 에서 * 버킷 * 을 선택합니다.

   Astra Control Service에서 사용할 수 있는 버킷 목록이 표시됩니다.

   다음 예제에서 볼 수 있듯이, Astra가 생성한 기본 버킷인 하나의 버킷만 사용할 수 있습니다.



추가 버킷을 추가합니다

클라우드 공급자에서 클러스터 관리를 시작한 후에는 언제든지 추가 버킷을 추가할 수 있습니다. 따라서 보호 정책을 생성할 때 버킷 중에서 선택하고 임시 백업 및 클론의 기본 버킷을 변경할 수 있습니다.

Astra Control Service를 사용하면 추가한 버킷을 제거할 수 없습니다.

무엇을 '필요로 할거야

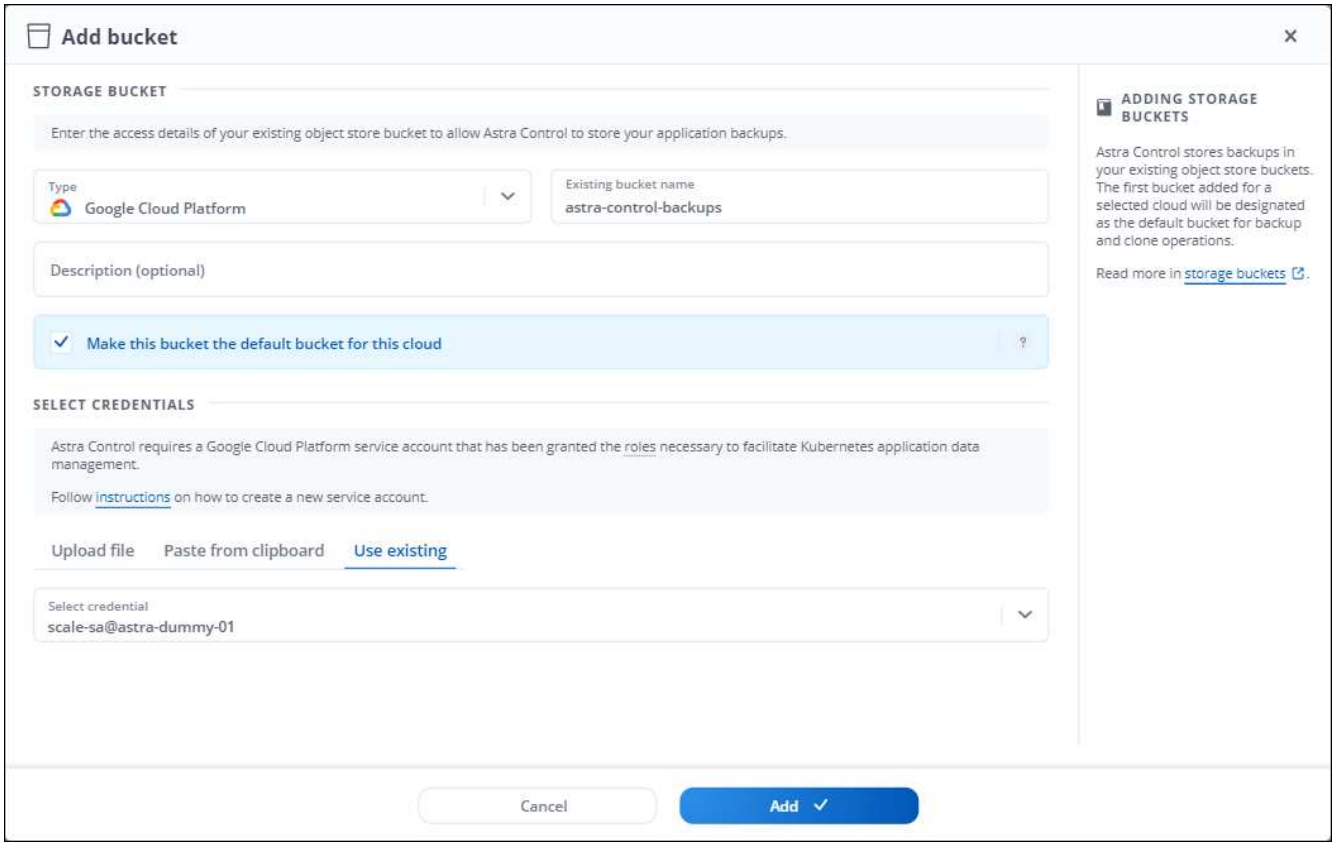
- 클라우드 공급자에 있는 기존 버킷의 이름입니다.
- Azure에 있는 버킷은 _Astra-backup-rg_라는 리소스 그룹에 속해야 합니다.

단계

1. 스토리지 관리 * 에서 * 버킷 * 을 선택합니다.
2. 추가 * 를 선택하고 프롬프트에 따라 버킷을 추가합니다.
   - * 유형 *: 클라우드 공급자를 선택하십시오.

     클라우드 공급자는 Astra Control Service가 해당 클라우드 공급자에서 실행 중인 클러스터를 관리하기 시작한 후에만 사용할 수 있습니다.
   - * 기존 버킷 이름 *: 버킷의 이름을 입력합니다.
   - * 설명 *: 버킷에 대한 설명을 선택적으로 입력합니다.
   - * 이 버킷을 이 클라우드의 기본 버킷으로 설정 *: 임시 백업 및 클론을 위한 기본 버킷으로 이 버킷을 사용할지 여부를 선택합니다.
   - * 자격 증명 선택 *: Astra Control Service에 버킷 관리에 필요한 권한을 제공하는 자격 증명을 선택합니다.

     다음은 Google Cloud Platform에서 새 버킷을 추가하는 방법을 보여주는 예입니다.



3. 버킷을 추가하려면 * 추가 * 를 선택합니다.

Astra Control Service는 추가 버킷을 추가합니다. 이제 보호 정책을 생성할 때 버킷을 선택할 수 있습니다.

기본 버킷을 변경합니다

Astra Control Service가 백업 및 클론에 사용해야 하는 기본 버킷을 변경합니다. 각 클라우드 공급자마다 고유한 기본 버킷이 있습니다.

Astra Control Service는 기존 백업에서 클론을 생성하도록 선택하지 않을 경우 임시 백업 및 임시 클론에 클라우드

공급자에 대한 기본 버킷을 사용합니다.

단계

1. 스토리지 관리 * 에서 * 버킷 * 을 선택합니다.
2. 편집할 버킷의 * Actions * 열에서 드롭다운 목록을 선택합니다.
3. 이 버킷을 이 클라우드의 기본 버킷으로 설정 * 을 선택합니다.
4. Update * 를 선택합니다.

계정을 관리합니다

대금 청구를 설정합니다

Astra Control의 무료 플랜을 사용하면 최대 10개의 앱을 계정에서 관리할 수 있습니다. 10개 이상의 앱을 관리하려면 무료 요금에서 프리미엄 요금제로 업그레이드하여 청구서를 설정해야 합니다.

대금 청구 개요

Astra Control Service와 관련된 비용에는 두 가지 유형이 있습니다. Astra Control Service에는 NetApp이 비용을 청구하며, 영구 볼륨 및 오브젝트 스토리지에 대해서는 클라우드 공급자가 비용을 청구합니다.

Astra Control 서비스 청구

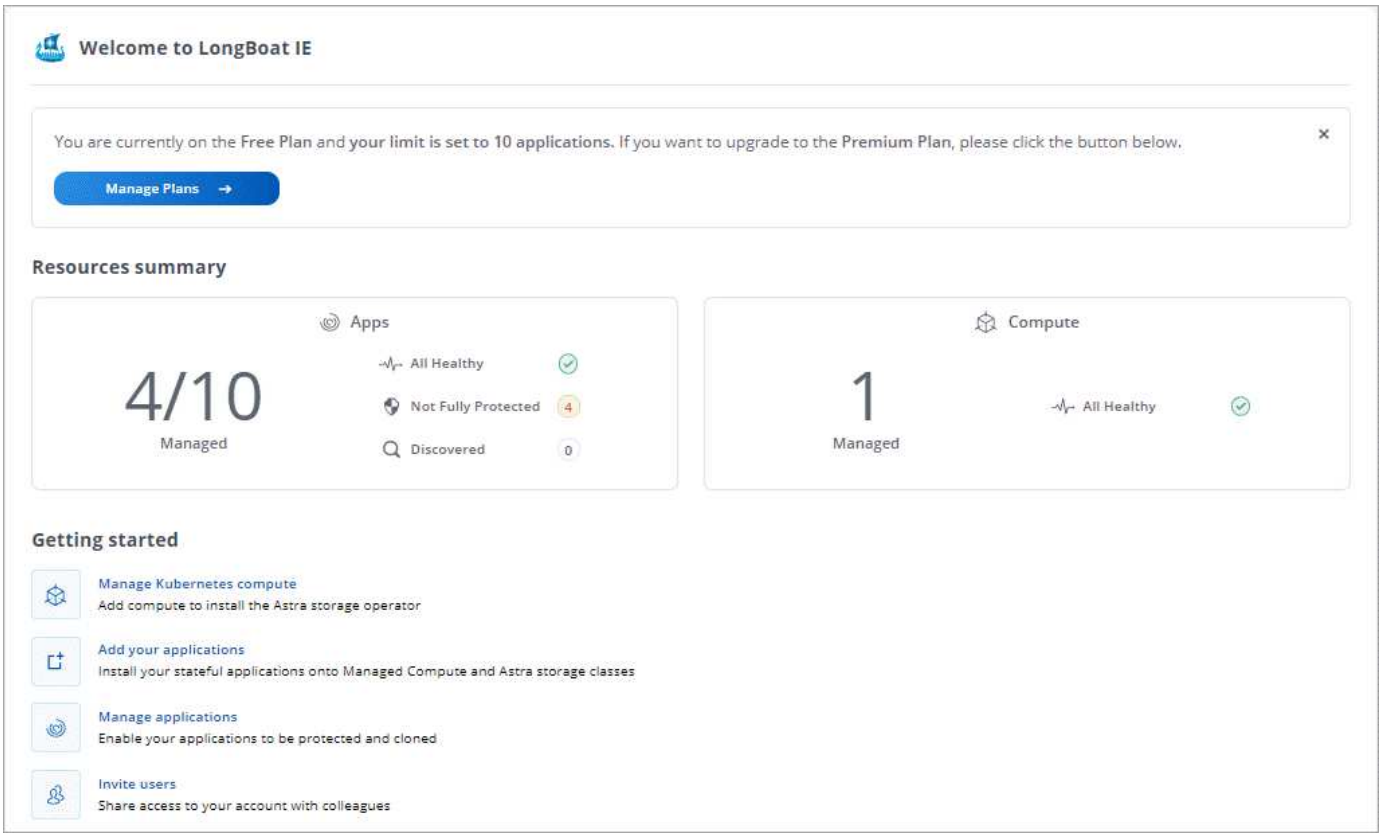
Astra Control Service는 다음과 같은 세 가지 계획을 제공합니다.

무료 플랜
: 최대 10개의 앱을 무료로 관리할 수 있습니다.

프리미엄 페이고
: 앱당 분당 $.005의 속도로 앱을 무제한으로 관리할 수 있습니다.

프리미엄 구독
: 아직 Astra Control 계정이 없는 경우 Premium Subscription을 구매하면 자동으로 Astra Control 계정이 생성됩니다. 기존 무료 플랜이 있는 경우 프리미엄 구독으로 자동 변환됩니다.

Astra Control 계정을 만들면 무료 플랜에 자동으로 등록됩니다. Astra Control의 대시보드에는 현재 관리하고 있는 앱 10개 중 허용되는 무료 앱 수가 표시됩니다.



11번째 앱을 관리하려고 하면 Astra Control에서 무료 요금제 한도에 도달했음을 알립니다. 그런 다음 무료 플랜에서 프리미엄 플랜으로 업그레이드하라는 메시지가 표시됩니다.

언제든지 프리미엄 요금제로 업그레이드할 수 있습니다. 업그레이드 후 Astra Control은 계정에 있는 _ALL_MANDATED 앱에 대해 귀하에게 요금을 부과하기 시작합니다. 처음 10개 앱은 무료 요금제에 머무르고 있지 않습니다.

Google Cloud 청구

Astra Control Service를 사용하여 GKE 클러스터를 관리할 경우, 영구 볼륨은 NetApp Cloud Volumes Service에 의해 백업되고 앱의 백업은 Google 클라우드 스토리지 버킷에 저장됩니다.

- "Cloud Volumes Service에 대한 가격 세부 정보를 봅니다".

  Astra Control Service는 모든 서비스 유형과 서비스 수준을 지원합니다. 사용하는 서비스 유형은 에 따라 다릅니다 "Google Cloud 지역".

- "Google Cloud 스토리지 버킷의 가격 세부 정보를 확인하십시오".

Microsoft Azure 청구

Astra Control Service를 사용하여 AKS 클러스터를 관리할 경우 영구 볼륨은 Azure NetApp Files에 의해 백업되고 앱 백업은 Azure Blob 컨테이너에 저장됩니다.

- "Azure NetApp Files에 대한 가격 세부 정보를 봅니다".
- "Microsoft Azure Blob 스토리지의 가격 세부 정보를 봅니다".

중요 참고 사항

- 귀하의 청구 계획은 Astra Control 계정입니다.

  계정이 여러 개인 경우 각 계정마다 자체 청구 계획이 있습니다.

- Astra Control 청구서에는 Kubernetes 앱 관리에 대한 비용이 포함되어 있습니다. 영구 볼륨의 스토리지 백엔드에 대해서는 클라우드 공급자가 별도로 요금을 부과합니다.

  "Astra Control 가격에 대해 자세히 알아보십시오".

- 각 청구 기간은 해당 월의 마지막 날에 종료됩니다.
- 프리미엄 요금제의 경우 무료 요금제로 다운그레이드할 수 없습니다.

무료 요금제의 프리미엄 페이고 요금제로 업그레이드하세요

언제든지 결제 계획을 업그레이드하면 Astra Control에서 10개 이상의 앱을 관리할 수 있습니다. 유효한 신용 카드만 있으면 됩니다.

단계

1. 계정 * 을 선택한 다음 * 청구 * 를 선택합니다.
2. 요금제 * 에서 * Premium PayGo * 로 이동하여 * 지금 업그레이드 * 를 선택합니다.
3. 유효한 신용 카드에 대한 결제 세부 정보를 제공하고 * 프리미엄 요금제로 업그레이드 * 를 선택하십시오.

   Astra Control은 신용 카드 만료일이 다가오면 이메일로 발송합니다.

이제 10개 이상의 앱을 관리할 수 있습니다. Astra Control은 현재 관리하고 있는 모든 앱에 대해 귀하에게 요금을 부과하기 시작합니다.

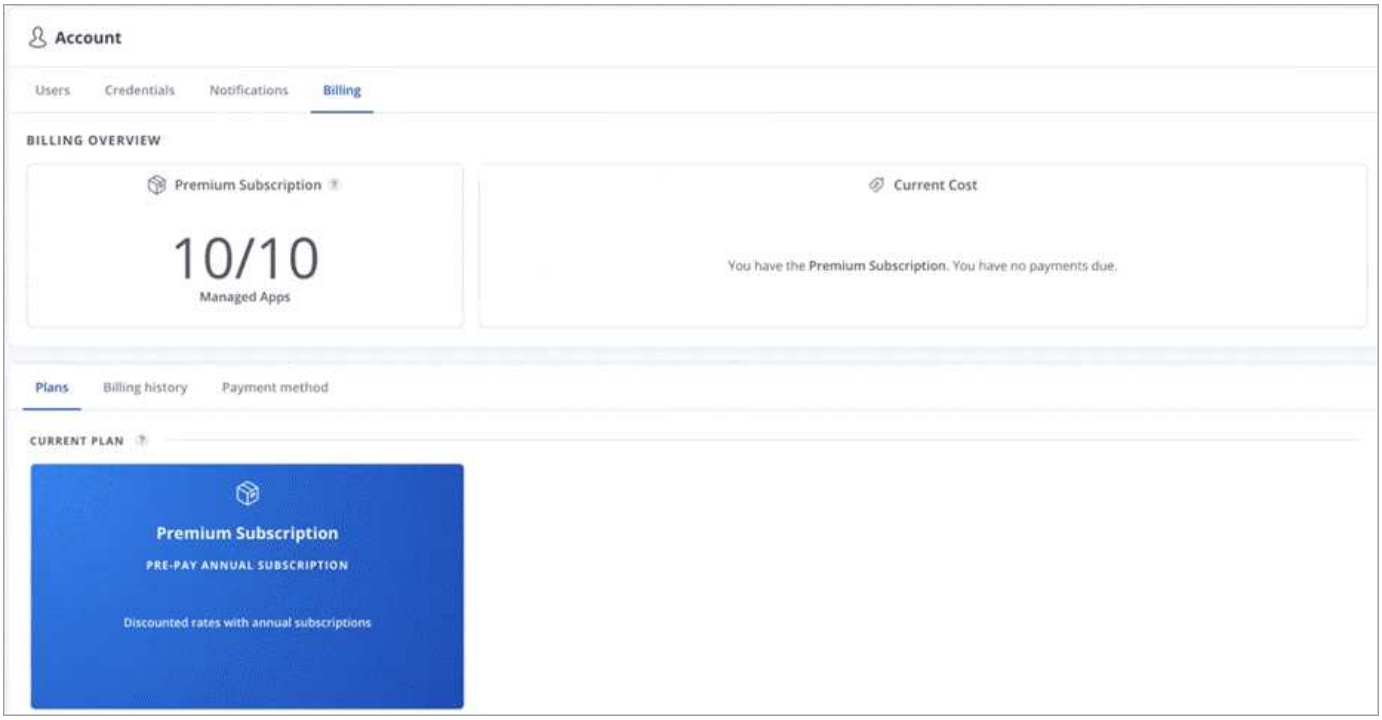
무료 요금제로부터 프리미엄 요금제로 업그레이드하세요

NetApp 세일즈 팀에 연락하여 연간 구독과 함께 할인된 요금으로 사전 결제할 수 있습니다.

단계

1. 계정 * 을 선택한 다음 * 청구 * 를 선택합니다.
2. 요금제 * 에서 * 프리미엄 이용 * 으로 이동하여 * 판매 연락처 * 를 선택합니다.
3. 프로세스를 시작할 수 있도록 세일즈 팀에 세부 정보를 제공합니다.

NetApp 세일즈 담당자가 구매 주문서를 처리하기 위해 연락을 드릴 것입니다. 주문이 완료되면 Astra Control이 청구 탭에 현재 계획을 반영합니다.



현재 비용 및 청구 기록을 봅니다

Astra Control은 현재 월간 비용 및 앱별 상세 청구 내역을 보여 줍니다.

단계

1. 계정 * 을 선택한 다음 * 청구 * 를 선택합니다.

   현재 비용이 청구 개요 아래에 표시됩니다.

2. 앱별 청구 내역을 보려면 * 청구 내역 * 을 선택합니다.

   Astra Control은 각 앱의 사용 시간 및 비용을 표시합니다. 사용 시간은 Astra Control이 청구 기간 동안 앱을 관리하는 시간(분)입니다.

3. 드롭다운 목록을 선택하여 이전 달을 선택합니다.

Premium PayGo의 신용 카드를 변경합니다

필요한 경우 Astra Control이 청구하기 위해 파일에 가지고 있는 신용 카드를 변경할 수 있습니다.

단계

1. 계정 > 청구 > 결제 방법 * 을 선택합니다.
2. 구성 아이콘을 선택합니다.
3. 신용 카드를 수정합니다.

사용자를 초대하고 제거합니다

Astra Control 계정에 가입하도록 사용자를 초대하고 더 이상 계정에 액세스할 수 없는 사용자를 제거합니다.

사용자를 초대합니다

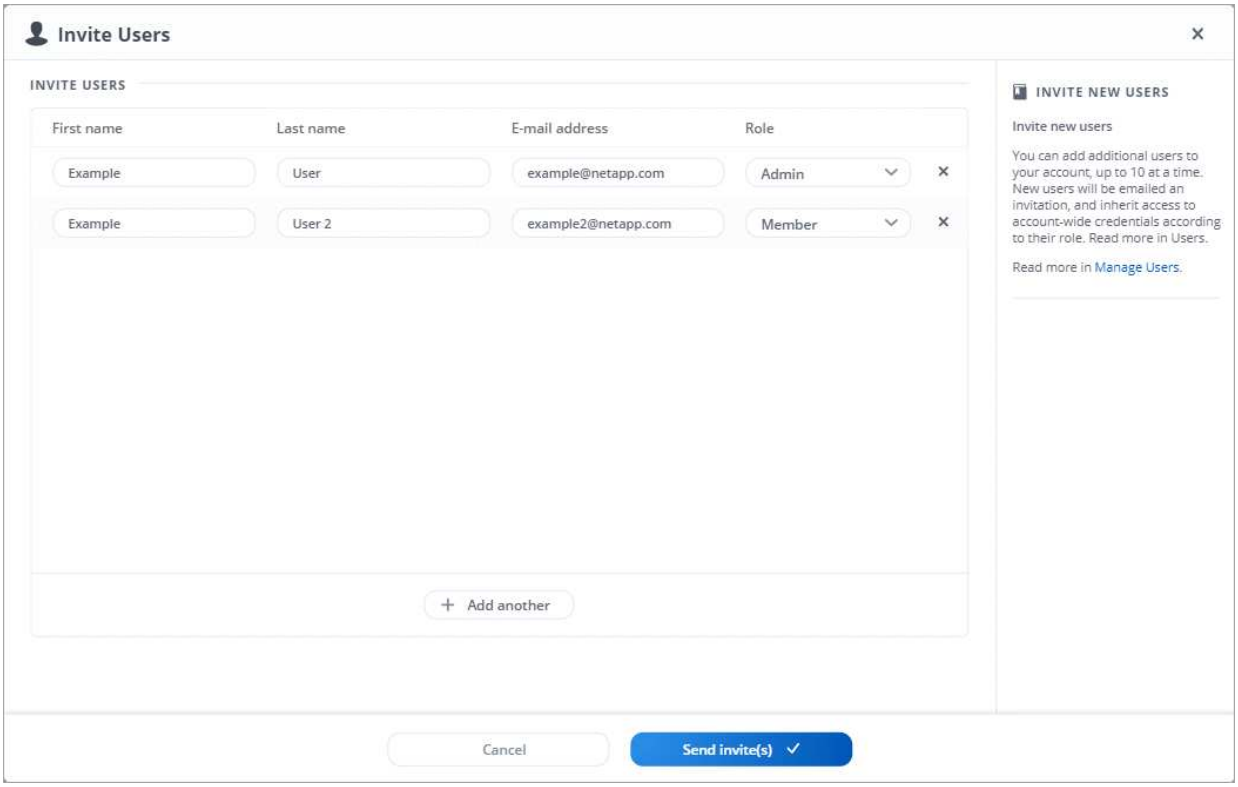
계정 소유자와 관리자는 다른 사용자를 Astra Control 계정에 가입하도록 초대할 수 있습니다.

단계

1. 사용자에게 가 있는지 확인합니다 "Cloud Central 로그인".
2. 계정 * 을 선택합니다.
3. 사용자 * 탭에서 * + 사용자 초대 * 를 선택합니다.
4. 사용자의 이름, 이메일 주소 및 역할을 입력합니다.

   다음 사항에 유의하십시오.

   - 이메일 주소는 사용자가 Cloud Central에 등록하는 데 사용한 이메일 주소와 일치해야 합니다.
   - 각 역할은 다음과 같은 권한을 제공합니다.
     - 소유자 * 는 관리자 권한을 가지며 계정을 삭제할 수 있습니다.
     - Admin * 은 구성원 권한을 가지며 다른 사용자를 초대할 수 있습니다.
     - 회원 * 은 앱과 클러스터를 완벽하게 관리할 수 있습니다.
     - Viewer * 는 리소스를 볼 수 있습니다.



5. 초대 보내기 * 를 선택합니다.

사용자는 계정에 가입하도록 초대하는 이메일을 받게 됩니다.

사용자의 역할을 변경합니다

계정 소유자는 모든 사용자의 역할을 변경할 수 있고 계정 관리자는 관리자, 구성원 또는 뷰어 역할을 가진 사용자의 역할을 변경할 수 있습니다.

단계

1. 계정 * 을 선택합니다.
2. 사용자 * 탭의 * 역할 * 열에 있는 드롭다운 목록을 선택합니다.
3. 새 역할을 선택한 다음 메시지가 표시되면 * 역할 변경 * 을 선택합니다.

Astra Control은 선택한 새 역할에 따라 사용자의 권한을 업데이트합니다.

사용자를 제거합니다

계정 소유자는 언제든지 계정에서 다른 사용자를 제거할 수 있습니다.

단계

1. 계정 * 을 선택합니다.
2. 사용자 * 탭에서 제거할 사용자를 선택합니다.
3. Actions * 를 선택하고 * Remove user/s * 를 선택합니다.
4. 메시지가 표시되면 사용자 이름을 입력하여 삭제를 확인한 다음 * 예, 사용자 제거 * 를 선택합니다.

Astra Control은 사용자를 계정에서 제거합니다.

자격 증명을 추가 및 제거합니다

언제든지 계정에서 클라우드 공급자 자격 증명을 추가 및 제거할 수 있습니다. Astra Control은 이러한 자격 증명을 사용하여 Kubernetes 클러스터, 클러스터에 있는 앱을 검색하고 대신 리소스를 프로비저닝합니다.

Astra Control의 모든 사용자는 동일한 자격 증명 세트를 공유합니다.

자격 증명을 추가합니다

Astra Control에 자격 증명을 추가하는 가장 일반적인 방법은 클러스터를 관리하는 것이지만 계정 페이지에서 자격 증명을 추가할 수도 있습니다. 그러면 추가 Kubernetes 클러스터를 관리할 때 자격 증명을 선택할 수 있습니다.

무엇을 '필요로 할거야

- GKE의 경우 필요한 권한이 있는 서비스 계정에 대한 서비스 계정 키 파일이 있어야 합니다. "서비스 계정 설정 방법에 대해 알아보십시오".
- AKS의 경우 서비스 보안 주체를 생성할 때 Azure CLI의 출력이 포함된 JSON 파일이 있어야 합니다. "서비스 보안 주체를 설정하는 방법에 대해 알아봅니다".

  JSON 파일에 추가하지 않은 경우 Azure 구독 ID도 필요합니다.

단계

1. 계정 > 자격 증명 * 을 선택합니다.
2. 자격 증명 추가 * 를 선택합니다.

3. Microsoft Azure * 또는 * Google Cloud Platform * 을 선택합니다.
4. Astra Control의 다른 자격 증명과 구별되는 자격 증명의 이름을 입력합니다.
5. 필요한 자격 증명을 입력합니다.
   a. * Microsoft Azure *: JSON 파일을 업로드하거나 클립보드에서 해당 JSON 파일의 내용을 붙여넣어 Azure 서비스 교장에게 자세한 정보를 Astra Control에 제공합니다.

      JSON 파일에는 서비스 보안 주체를 생성할 때 Azure CLI의 출력이 포함되어야 합니다. 또한 구독 ID를 포함할 수 있으므로 Astra Control에 자동으로 추가됩니다. 그렇지 않으면 JSON을 제공한 후 ID를 수동으로 입력해야 합니다.

   b. * Google Cloud Platform *: 파일을 업로드하거나 클립보드의 콘텐츠를 붙여넣어 Google Cloud 서비스 계정 키 파일을 제공합니다.
6. 자격 증명 추가 * 를 선택합니다.

이제 Astra Control에 클러스터를 추가할 때 자격 증명을 선택할 수 있습니다.

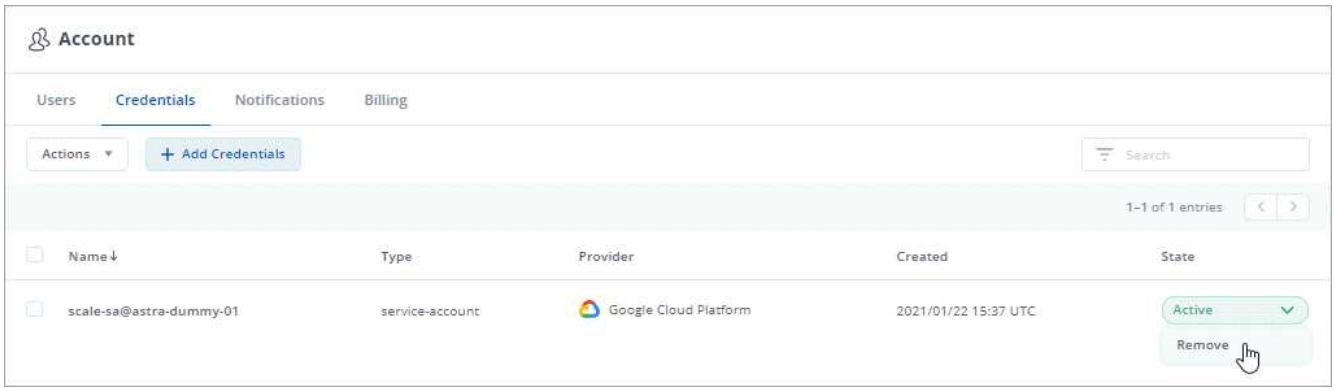
자격 증명을 제거합니다

언제든지 계정에서 자격 증명을 제거합니다. 자격 증명은 이후에 제거해야 합니다 "모든 클러스터 관리를 취소합니다".

> Astra Control에 추가하는 첫 번째 자격 증명 세트는 항상 사용 중입니다. Astra Control은 자격 증명을 사용하여 백업 버킷에 인증하기 때문입니다. 이러한 자격 증명을 제거하지 않는 것이 좋습니다.

단계

1. 계정 > 자격 증명 * 을 선택합니다.
2. 제거할 자격 증명에 대한 * 상태 * 열의 드롭다운 목록을 선택합니다.
3. 제거 * 를 선택합니다.



4. 삭제할 자격 증명의 이름을 입력한 다음 * 예, 자격 증명 제거 * 를 선택합니다.

Astra Control은 계정에서 자격 증명을 제거합니다.

계정 활동을 봅니다

Astra Control 계정의 활동에 대한 세부 정보를 볼 수 있습니다. 예를 들어, 새 사용자를 초대하거나, 클러스터를 추가하거나, 스냅샷을 생성할 때 사용할 수 있습니다. 계정 활동을 CSV

파일로 내보낼 수도 있습니다.

Astra Control의 모든 계정 활동을 보는 단계

1. Activity * 를 선택합니다.
2. 필터를 사용하여 활동 목록의 범위를 좁히거나 검색 상자를 사용하여 원하는 항목을 정확하게 찾을 수 있습니다.
3. CSV로 내보내기 * 를 선택하여 계정 활동을 CSV 파일로 다운로드합니다.

특정 앱의 계정 활동을 보는 단계

1. 응용 프로그램 * 을 선택한 다음 앱 이름을 선택합니다.
2. Activity * 를 선택합니다.

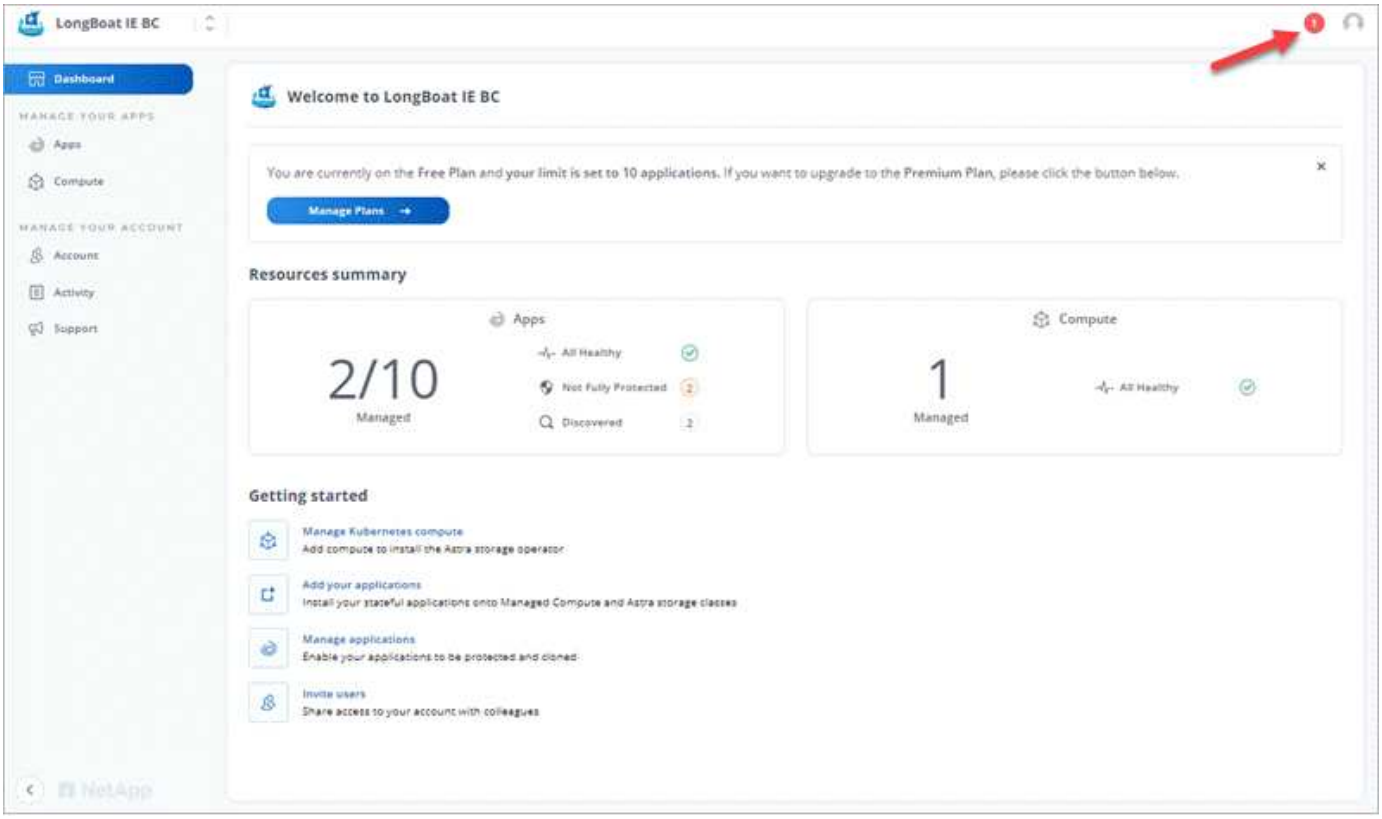
클러스터의 계정 활동을 확인하는 단계입니다

1. 클러스터 * 를 선택한 다음 클러스터 이름을 선택합니다.
2. Activity * 를 선택합니다.

알림을 보고 관리합니다

Astra Control은 작업이 완료되거나 실패했을 때 알려줍니다. 예를 들어, 앱 백업이 성공적으로 완료되면 알림이 표시됩니다.

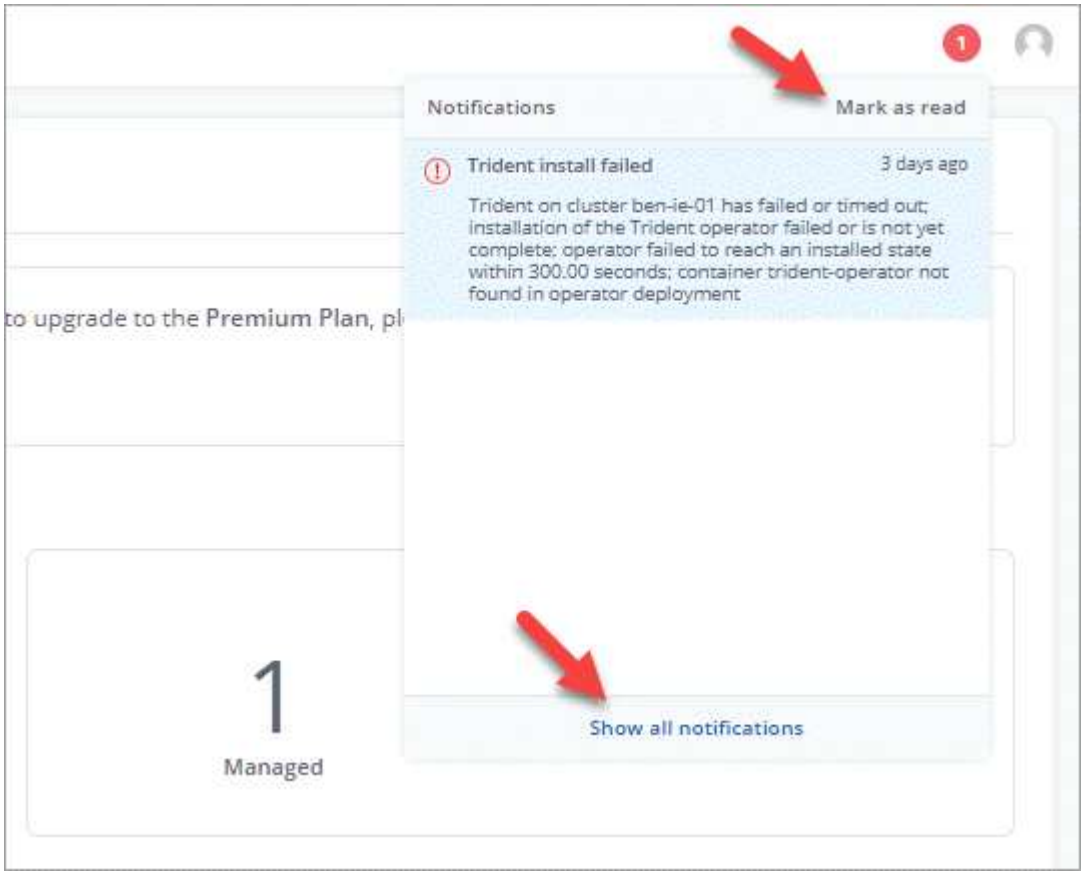
인터페이스의 오른쪽 상단에서 읽지 않은 알림 수를 사용할 수 있습니다.



이러한 알림을 보고 읽은 상태로 표시할 수 있습니다. 이렇게 하면 읽지 않은 알림을 지우는 것이 편리합니다.

단계

1. 오른쪽 상단에서 읽지 않은 알림 수를 선택합니다.



2. 알림을 검토한 후 * 읽은 상태로 표시 * 또는 * 모든 알림 표시 * 를 선택합니다.

   모든 알림 표시 * 를 선택한 경우 알림 페이지가 로드됩니다.

3. 알림 * 페이지에서 알림을 보고 읽음으로 표시할 알림을 선택하고 * 작업 * 을 선택한 다음 * 읽음으로 표시 * 를 선택합니다.

계정을 닫습니다

Astra Control 계정이 더 이상 필요하지 않으면 언제든지 계정을 종료할 수 있습니다.

> 계정을 닫으면 Astra Control이 자동으로 생성한 버킷이 자동으로 삭제됩니다.

단계

1. "모든 앱 및 클러스터 관리를 취소합니다".
2. "Astra Control에서 자격 증명을 제거합니다".
3. 계정 > 청구 > 결제 방법 * 을 선택합니다.
4. 계정 닫기 * 를 선택합니다.
5. 계정 이름을 입력하고 확인 을 클릭하여 계정을 닫습니다.

앱 및 클러스터 관리를 취소합니다

Astra Control에서 더 이상 관리하지 않으려는 응용 프로그램 또는 클러스터를 제거합니다.

앱 관리를 중지합니다

Astra Control에서 더 이상 백업, 스냅샷 또는 클론 복제하지 않을 애플리케이션 관리를 중지합니다.

- 기존 백업 및 스냅샷이 삭제됩니다.
- 애플리케이션과 데이터는 사용 가능한 상태로 유지됩니다.

단계

1. 응용 프로그램 * 을 선택합니다.
2. 더 이상 관리하지 않을 앱의 확인란을 선택합니다.
3. Action * 드롭다운을 선택하고 * Unmanage application/s * 를 선택합니다.
4. 앱 관리를 해제할지 확인한 다음 * 예, 응용 프로그램 관리 취소 * 를 선택합니다.

Astra Control은 앱 관리를 중지합니다.

클러스터 관리를 중지합니다

Astra Control에서 더 이상 관리하지 않으려는 클러스터 관리를 중지합니다. 모범 사례로서, GCP를 통해 클러스터를 삭제하기 전에 Astra Control에서 클러스터를 삭제하는 것이 좋습니다.

- 이 작업을 수행하면 Astra Control에서 클러스터를 관리할 수 없습니다. 클러스터 구성을 변경하지 않고 클러스터를 삭제하지 않습니다.
- Astra Trident가 클러스터에서 제거되지 않습니다. "Astra Trident를 제거하는 방법을 알아보십시오".

단계

1. 클러스터 * 를 선택합니다.
2. 더 이상 관리하지 않으려는 클러스터의 확인란을 선택합니다.
3. Actions * 드롭다운을 선택하고 * Unmanage * 를 선택합니다.
4. 클러스터 관리를 해제할지 확인한 다음 * 예, 클러스터 관리 취소 * 를 선택합니다.

Astra Control은 클러스터 관리를 중지합니다.

클라우드 공급자에서 클러스터 삭제

NetApp 스토리지 클래스에 상주하는 PV(영구적 볼륨)가 있는 Kubernetes 클러스터를 삭제하려면 먼저 아래 방법 중 하나를 따라 PVC(영구 볼륨 클레임)를 삭제해야 합니다. 클러스터를 삭제하기 전에 PVC 및 PV를 삭제하면 클라우드 공급자로부터 예상치 못한 청구서를 받지 않게 됩니다.

- * 방법 #1 *: 클러스터에서 애플리케이션 워크로드 네임스페이스를 삭제합니다. Trident 네임스페이스를_삭제하지_마십시오.
- * 방법 #2 *: PVC와 POD를 삭제하거나 PVS가 마운트된 구축을 삭제합니다.

Astra Control에서 Kubernetes 클러스터를 관리할 경우, 해당 클러스터의 애플리케이션은 Azure NetApp Files 또는 Cloud Volumes Service를 영구 볼륨의 스토리지 백엔드로 사용합니다. PVS를 먼저 제거하지 않고 클라우드 공급자에서 클러스터를 삭제하는 경우 백엔드 볼륨은 클러스터와 함께 _NOT_DELETED가 됩니다.

위의 방법 중 하나를 사용하면 클러스터에서 해당 PVS가 삭제됩니다. 삭제하기 전에 클러스터의 NetApp 스토리지 클래스에 PVS가 없는지 확인하십시오.

클러스터를 삭제하기 전에 영구 볼륨을 삭제하지 않은 경우 Cloud Volumes Service for Google Cloud 또는 Azure NetApp Files에서 백엔드 볼륨을 수동으로 삭제해야 합니다.

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.